Fault to Forge: Fault Assisted Forging Attacks on LESS Signature Scheme

Puja Mondal[1], Supriya Adhikary[1], Suparna Kundu[2], Hikaru Nishiyama[3], Daisuke Fujimoto[3], Yuichi Hayashi[3], and Angshuman Karmakar[1]

[1] Department of Computer Science and Engineering, IIT Kanpur, India
{pujamondal,adhikarys,angshuman}@cse.iitk.ac.in
[2] COSIC, KU Leuven, Kasteelpark Arenberg 10, Bus 2452, B-3001 Leuven-Heverlee, Belgium
suparna.kundu@esat.kuleuven.be
[3] Nara Institute of Science and Technology, Ikoma, Japan
{nishiyama.hikaru.na1,fujimoto,yu-ichi}@is.naist.jp

Abstract. LESS is a digital signature scheme that is currently in the second round of the National Institute of Standards and Technology's (NIST's) ongoing call for standardization of additional post-quantum signature schemes. Recently, the ZKfault attack by Mondal et al. (Asiacrypt 2024) showed several attack surfaces on the first version of LESS (LESS-v1) that can be exploited using different fault attacks. However, the designers of LESS have updated the scheme in the second round to improve efficiency and signature size. The fault attacks in ZKfault are not directly applicable to this new version of LESS (LESS-v2). The physical security of LESS-v2 is still unexplored.
In this work, we analyze the new structure of the LESS-v2 signature scheme and propose a method for universal forgery. One crucial observation is that, for LESS-v2, an adversary does not require the full signing key, but some other secret-related information, namely, a permutation of secret monomial matrices. We assume an adversary can use execution interruption techniques, such as fault attacks, to recover this extra information. We have analyzed multiple such attack surfaces in the design of LESS-v2, and using fault injection, we can recover secret-related information. We showed the average number of required faults for two particular fault models to recover the secret equivalent component and observed that, for most parameters, we need only 1 faulted signature, and at most 1088. Our attacks rely on simple and standard fault models. We demonstrated these using both simulation and a simple experimental setup.

Keywords: Post-quantum cryptography · Digital signature · Code-based· Zero-Knowledge based · Fault attacks · LESS · LESS-v2 · Chip-whisperer

1 Introduction

LESS digital signature. The Linear Equivalence Signature Scheme (LESS) [5,4] is a code-based post-quantum digital signature scheme. It is currently one of the 14 candidate schemes in the second round of the ongoing National Institute of Standards and Technology (NIST)'s additional post-quantum signature standardization process [26]. LESS is also unique in the sense that, unlike other code-based schemes, it is not

based on the traditional Syndrome Decoding Problem (SDP). Rather LESS has been constructed by applying a Fiat-Shamir [15] transformation to a 3-pass Sigma protocol. The security of LESS in round 1 was based on the hardness of proving the knowledge of an isometry between a pair of linear codes or linear equivalence problem (LEP). Interestingly, it is also the first signature scheme based on this LEP-hard problem [11,5]. In order to improve efficiency and reduce signature size, the authors of LESS updated the underlying hard problem from LEP to Canonical Form Linear Equivalence Problem (CF-LEP) [13] in the second round submission of NIST's additional standardization effort. This unique construction enables LESS to achieve smaller public key and public key+signature sizes with respect to many post-quantum signature schemes, such as CROSS [6], Mirath [2], SPHINCS+ [10], Classic McEliece [3], etc.

Goldreich-Goldwasser-Micali (GGM) Tree. The GGM Tree construction was originally proposed by Goldreich et al. [17], providing a method to construct a pseudorandom function (PRF) from any pseudorandom generator. In a Fiat-Shamir transformed multi-round signature scheme such as LESS, the verifier presents a challenge $c_i \in \{0,1\}$. Based on this challenge, the signer reveals a specific isometry that either 'opens' the commitment or shows its relation to the secret key. The Zero-Knowledge (ZK) property strictly requires that the adversary never sees the responses for both $c_i = 0$ and $c_i = 1$ for the same commitment. The signer must generate t independent seeds $(seed_0,...,seed_{t-1})$ to derive the randomness for each parallel instance of the Sigma protocol. Transmitting these seeds directly scales the signature size linearly with n. To mitigate this, LESS employs a tree-based seed derivation mechanism, specifically the GGM, to compress the signature by deriving all session randomness from a single master seed. The GGM tree makes the signature size logarithmic with respect to the number of rounds. In many modern ZKP-based and MPC-in-the-Head-based signature schemes, such as CROSS [28], Picnic [12], SDitH [22], Mirath [2], FAEST [8], etc., GGM tree, serve similar purposes. Therefore, in these digital signature constructions, this tree serves a dual purpose: compressing the signature size and enforcing the ZK property.

In its original context [17], and even in early blockchain or verifiable random function (VRF) applications, the primary goal of the seed tree was to provide a compact representation of a pseudorandom sequence. In those scenarios, a physical attack on the tree generation might result in a localized denial-of-service or a predictable output, but it rarely compromises the foundational long-term secret of the system. In these contexts, physical attacks such as fault injection attacks (FIA), side-channel attacks (SCA), or micro-architectural attacks (MA) were of secondary concern. However, in the context of ZKP-based digital signatures, if a fault can force the GGM expansion to reuse seeds or skip the "hiding" logic, or an SCA can leak internal hidden states of the tree, resulting in the violation of the Zero-Knowledge property. In the LESS signature, the signer publishes the seeds corresponding to all those challenges where the challenge value is zero. Therefore, the signer computes a flag tree that identifies only those nodes in the generated GGM tree that can generate the seeds corresponding to zero challenge value. Since the incorrect generation of the flag tree can reveal the seeds that are supposed to be hidden, an adversary can target the generation of this flag tree to gain some information. This makes the whole process of GGM tree the single point of failure in a signature algorithm for the implementation's resistance against physical adversaries.

Related work. In this work, our focus is on the cryptanalysis of the LESS Signature in the presence of additional information obtained via methods such as side-channel traces or fault injection. Such analyzes are becoming increasingly common, especially for cryptographic schemes deployed widely in the public domain, such as digital signature schemes [21,7,16,18,24,14,19].

In related work, Mondal et al. proposed a fault attack on LESS version 1 (LESS-v1) [24]. In this work, the authors targeted the response publishing part of LESS-v1, which is used for signature sizes. LESS-v1 used a complete binary tree to decide which intermediate seed of the Goldreich-Goldwasser-Micali (GGM) tree will be published and which seed will remain hidden from the verifier. In this paper, the authors have identified multiple fault locations and approaches in this complete binary tree and also provided a countermeasure against these fault attacks. In the paper [14], Czuprynko et al. have presented a correlation power analysis on the multiplication operation of ephemeral monomial matrices with the public key in LESS-v, although they did not provide any countermeasure against this attack.

Since the publication of these attacks, the design of LESS has been updated; we call this updated signature LESS-v2 or simply the LESS signature. The designers transitioned from a complete GGM tree and the corresponding complete binary decision tree to an incomplete setup to optimise speed and reduce stack memory usage. Since the attack location (complete binary decision tree) has entirely changed in LESS-v2, the attack described in the ZKFault paper does not apply to LESS-v2. Additionally, in LESS-v2, the authors have changed their response part to reduce their signature size. So the secret-extraction process proposed in the ZKFault paper will not work with LESS-v2.

In paper [19], Jendral et al. presented a side-channel and fault-analysis of the VOLEitH signature scheme FAEST [8]. The authors targeted the GGM tree generation process. They have shown that by injecting a fault attack into the PRG function (children seed node generation from the root seed), two identical children seed nodes can be generated. Following this observation, they recover the secret information. The authors claimed that the same attack strategy can be applied to the LESS signature too. This fault location is different from our fault location.

Most recently, Huang et al. have presented a fault attack analysis [18] on the LESS-v2 signature. They explained their attack model at the array-computation step, which occurs just before constructing the incomplete binary decision tree in the LESS-v2 signature. Although they have not provided a countermeasure for the fault they presented, we think the targeted fault location is an extra step in the LESS-v2 algorithm and can be removed entirely, which will stop their attack. We explained this in Section 4.1. Additionally, they have first computed secret monomial matrices. Although in our work, we have shown that only the permutations corresponding to the secret equivalent monomial matrix are sufficient for signature forgery.

1.1 Our contributions and approach

In this work, we present the first physical attack on LESS-v2 using fault injection. The core idea is to trick the signature algorithm to leak information corresponding to both challenge values 0 and 1. As in normal execution, this never happens; we turn to fault injection to disrupt the execution flow of the GGM tree construction.

We demonstrate that the GGM tree construction method used in LESS-v2 creates a significantly large number of fault-injection surfaces that lead to *universal* forgery. Our specific contributions are

Universal forgery through recovery of permutations: In LESS-v2, the signing key is a small seed value $\texttt{seed}_{\texttt{sk}}$. This seed is expanded through an `XOF` to generate $s-1$ monomial matrices. In Section. 3, we deduced that an adversary only needs $(s-1)$ permutations corresponding to $(s-1)$ monomial matrices to universally forge a signature. This reduction significantly lowers the bar for an adversary. Since we are able to forge any arbitrary message, this is as good as recovering the signing key. However, in this scenario, the bar is much lower as we do not need to recover the signing key $\texttt{seed}_{\texttt{sk}}$, which is hidden behind an `XOF`. Further, we formulate a method for the extraction of the targeted permutation from faulted signatures. We believe this formulation can be utilized by other physical attacks as well.

Systematic uncovering of attack surfaces: Under normal circumstances, a properly generated signature does not leak any information related to the secret or the above-mentioned permutation. We did a thorough analysis and uncovered a multitude of attack surfaces and designed bespoke strategies to recover this secret-related information using fault injections. More specifically, LESS-v2 uses an incomplete binary decision tree to compress the signature size. This binary tree is used to decide which part (some ephemeral part) should be public and which part should be hidden. In Section 4, we analyze this publishing part, starting from the decision tree generation to the publishing process of using this tree. We identified ten locations that will serve our purpose to find our requirement.

Simple and minimalist fault model: A core objective of this work is to decouple the efficacy of the attack from the sophistication of the experimental apparatus. We have shifted the complexity from hardware precision and expertise to rigorous mathematical analysis. This has helped us to realize our attack on a simple experimental apparatus. We have discussed our various fault attack strategies in Section 4.2, 4.3, and 4.4. Most of our attacks work using only the minimalist instruction skip fault model. Unlike other fault models, such as bit-flip, stuck-at-one, stuck-at-zero, single-bit fault, etc., which depend on specific register widths or memory technologies, instruction skips are a universal phenomenon across virtually all modern microprocessors. Instruction skip can be implemented using simple voltage or clock glitching, which can be achieved with inexpensive entry-level hardware like the ChipWhisperer, unlike the other fault models, which require specialized equipment like laser or localized electro-magnetic fault injection setup and significant expertise to perform. In this manner, we have democratized our attack, which further increases its impact.

Quantitative analysis for fault estimation: As an added contribution, in Section 5, we have provided a quantitative analysis to deduce the expected amount of information that we can obtain from one faulted signature. Our analysis shows that this amount of information largely depends on the fault strategy. Nevertheless, it allows us to estimate the required number of faulted signatures to recover the secret equivalent information to forge the signature very accurately.

Robustness under noisy fault conditions: In any practical fault injection attack, the outcome is inherently probabilistic. The majority of fault injections typically either

fail to make the desired change in execution or memory location, or even when they are able to do so, their effect does not manifest in the ciphertext or signature, resulting in "silent" faults. We argue that the presence of a robust distinguisher is fundamental to the efficacy of any general fault attack. Without a mechanism to filter the output stream, the "signal" *i.e.* the secret-leaking signature is drowned in the "noise" of invalid or trivial outputs. Since we are aiming for a forgery attack, a straightforward approach to isolate a correctly recovered permutation from an incorrectly recovered permutation is to sign a message and verify it. However, for this strategy, we have to wait until the complete recovery of the permutation. In Section 6, we have shown that for some of our attacks, we can immediately isolate a useful signature from a noisy one, leading to a very efficient attack. Unfortunately, for the other attack strategies, we cannot detect this immediately, and we have to wait until the whole permutation has been recovered.
End-to-end attack simulation and practical experiments: Although we explain the fault attack methodology by assuming the fault assumption, i.e., in a theoretical framework, it can be realized on several commonly used faults. In Section 7, we have demonstrated this through a practical experiment on a CW308 UFO board with a Cortex-M4 (STM32F303) for the parameter set LESS-252-45 and using fault models 9 and 10. Our attack is end-to-end, but due to the memory limitations of our experimental setup, the full signing algorithm cannot run on the target device [20]. However, it is pretty straightforward to extrapolate our attack to a device that can run the complete signing algorithm. We have demonstrated this in our simulation code

2 Preliminaries

We denote $\mathbb{Z}_q$ as the ring of integers modulo q. Additionally, $\mathbb{F}_q$, $\mathbb{F}_q^k$ and $\mathbb{F}_q^{k\times n}$ represent the field with q elements, the set of all vectors of k length and $k\times n$ ordered matrices over the field $\mathbb{F}_q$ respectively. All scalars and a set of scalars are denoted by the lowercase letter and uppercase letters, respectively. The bold lowercase is used to denote the vectors over the field $\mathbb{F}_q$. Let $\boldsymbol{a}\in\mathbb{F}_q^k$. Then, we denote the i-th entry of the vector $\boldsymbol{a}$ by $\boldsymbol{a}[i]$. The i-th standard basis is denoted by $\boldsymbol{e}_i$ and is defined as the vector whose i-th element is one, and the remaining elements are zero. The matrices over the field $\mathbb{F}_q$ are represented by the bold uppercase letter. The $(i,\ j)$-th entry of a matrix $\boldsymbol{A}$ is denoted by $\boldsymbol{A}[i,\ j]$. The i-th row and j-th column of the matrix $\boldsymbol{A}$ are denoted by $\boldsymbol{A}[i,\ *]$ and $\boldsymbol{A}[*,\ j]$ respectively. Similarly, we use $\boldsymbol{A}[*,\ J]$ to denote the submatrix of $\boldsymbol{A}$ formed by selecting the columns indexed by $J=\{j_0,\ \cdots,\ j_{k-1}\}$, taken in order. The transpose and inverse matrices of a matrix $\boldsymbol{A}$ are denoted by $\boldsymbol{A}^T$ and $\boldsymbol{A}^{-1}$ respectively. The identity matrix of order n is denoted by $\boldsymbol{I}_n$, whose only diagonal elements are one, and the remaining elements are zero. A permutation π on the set $\mathbb{Z}_n$ is a bijective map on $\mathbb{Z}_n$. The set of all invertible matrices of order n over the field $\mathbb{F}_q$ is denoted by $GL_n(q)$. We denote a monomial matrix $\boldsymbol{A}$ of order n by $(\boldsymbol{u},\ \pi)$, where $\boldsymbol{u}$ is the a vector and π is a permutation. A partial permutation is a injective map and we denote at as π^* (upper star). The definition of monomial matrices, its properties, some elementary matrix operations are given in Appendix A.

2.1 LESS-v2

In this section, we briefly describe the signature algorithm of LESS-v2, focusing mainly on the functions required to explain our attack. The notation of other functions is

similar to the LESS documentation [4]. The key generation and Verification algorithms, along with the current parameter set of LESS-v2, are provided in Appendix B. The code parameters n, k, q, and s represent the code length, code dimension, prime modulus, and the number of secret monomial matrices, respectively. The parameter set *LESS-n-t* represents the signature scheme with code dimension n and length of the digest t. Also, w represents the digest's weight.

Algorithm 1 `LESS Signature`

Input: Message `msg` and secret key $\mathtt{sk} = \mathtt{seed}_{\mathtt{sk}}$.
Output: The signature τ of the message `msg`.

$\mathtt{state}_{\mathtt{sk}} \leftarrow \mathtt{XOF.init}(\mathtt{seed}_{\mathtt{sk}})$
$\mathtt{seed}_{\mathtt{pk}} \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$\boldsymbol{G}_0 \leftarrow \mathtt{SampleGenerator}(\mathtt{seed}_{\mathtt{pk}})$
$(\mathtt{seed}_1, \mathtt{seed}_2, ..., \mathtt{seed}_{s-1}) \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$\mathtt{salt} \xleftarrow{\$} \{0, 1\}^{2\lambda}$
$\mathtt{seed}_{\mathtt{tree}} \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$(\mathtt{seed}'_0, \cdots, \mathtt{seed}'_{2t-1}) \leftarrow \mathtt{BuildGGM}(\mathtt{seed}_{\mathtt{tree}}, \mathtt{salt})$
$(\mathtt{eseed}_0, ..., \mathtt{eseed}_{t-1}) \leftarrow \mathtt{SeedLeaves}(\mathtt{seed}')$
$\mathtt{state}_{\mathtt{blind}} \leftarrow \mathtt{XOF.init}(\mathtt{XOF}(\mathtt{state}_{\mathtt{sk}}))$
for $i = 0, ..., t-1$ **do**
    $\widetilde{\boldsymbol{Q}}_i = (\boldsymbol{v}_i, \widetilde{\pi}_i) \leftarrow \mathtt{SampleMonomial}(\mathtt{eseed}_i || \mathtt{salt} || i)$
    $\left(\widetilde{\mathbf{G}}_i, \widetilde{J}\right) \leftarrow \mathtt{RREF}(\boldsymbol{G}_0 \widetilde{\boldsymbol{Q}}_i^T)$
    $\widetilde{\pi}_i^* = (\widetilde{\pi}_i^{-1})_{|J}$
    $\widetilde{\boldsymbol{A}}_i = \widetilde{\boldsymbol{G}}_i[*, \widetilde{J}^c]$
    $\widetilde{\boldsymbol{B}}_i \leftarrow \mathtt{BLIND}(\mathtt{state}_{\mathtt{blind}}, \widetilde{\boldsymbol{A}}_i)$
    $\widetilde{\boldsymbol{C}}_i \leftarrow \mathtt{CF}(\widetilde{\boldsymbol{B}}_i)$
$\mathtt{cmt} \leftarrow \mathtt{HASH}(\widetilde{\boldsymbol{C}}_0, ..., \widetilde{\boldsymbol{C}}_{t-1} || \mathtt{salt} || \mathtt{msg})$
$\boldsymbol{b} = (\boldsymbol{b}[0], ..., \boldsymbol{b}[t-1]) \leftarrow \mathtt{SampleChallenge}(\mathtt{cmt})$
for $i = 0$ to $t-1$ **do**
    If $\boldsymbol{b}[i] = 0$ **then** $\boldsymbol{U}[\boldsymbol{i}] = 0$
    else $\boldsymbol{U}[\boldsymbol{i}] = 1$
$\mathtt{path} \leftarrow \mathtt{GGMPath}(\mathtt{seed}', \boldsymbol{U})$
$k = 0$
for $i = 0, ..., t-1$ **do**
    if $\boldsymbol{b}[i] \neq 0$ **then**
        $\boldsymbol{Q}_{\boldsymbol{b}[i]} = (\boldsymbol{u}_{\boldsymbol{b}[i]}, \pi_{\boldsymbol{b}[i]}) \leftarrow \mathtt{SampleMonomial}(\mathtt{seed}_{\boldsymbol{b}[i]})$
        $\pi^{*'} \leftarrow \mathtt{MonomialCompose}(\boldsymbol{Q}_{\boldsymbol{b}[i]}, \widetilde{\pi}_i^*)$
        $\mathtt{rsp}_k \leftarrow \mathtt{CosetRep}(\boldsymbol{Q}_{\mathtt{mult}}),\ k = k+1$
$\tau \leftarrow \left\{\mathtt{cmt}, \mathtt{salt}, \mathtt{path}, \{\mathtt{rsp}_i\}_{i \in \mathbb{Z}_w}\right\}$
Return the signature τ

Signature generation of LESS-v2: The signature algorithm, presented in Alg. 1, takes the message `msg` and the secret key $\mathtt{sk} = \boldsymbol{seed_{sk}}$ as input and outputs the signature τ. This algorithm computes the seed $\mathtt{seed}_{\mathtt{pk}}$ of the public matrix $\boldsymbol{G}_0$, $s-1$ secret seeds $\mathtt{seed}_i$, $i \in [1, s-1]$ of the corresponding secret monomial matrices $\boldsymbol{Q}_i$, the seed $\mathtt{seed}_{\mathtt{tree}}$ of the ephemeral secret, the seed $\mathtt{seed}_{\mathtt{blind}}$ of blind monomial matrices from the secret key $\mathtt{seed}_{\mathtt{sk}}$. Next, a random `salt` is sampled and used along with $\mathtt{seed}_{\mathtt{tree}}$ as input of `BuildGGM` function to generate the seeds $\mathtt{eseed}_i$ of the ephemeral monomial matrices $\widetilde{\boldsymbol{Q}}_i$.

In LESS-v2, an incomplete seed tree (GGM tree) is used instead of a complete GGM tree, as in the LESS-v1 signature algorithm for ephemeral key generation. So, here, the index relation of the parent node (say i) with its left and right child nodes is different from the complete tree, where it should be $2i+1$ and $2i+2$. To relate the chil-

dren nodes to their parent node, LESS-v2 defines the total number of nodes and leaf nodes in each level, the starting indices of the leaf nodes in each level, and the offsets for each level of the GGM tree. The incomplete GGM tree generation is performed using `BuildGGM` algorithm (presented in Appendix D, Alg. 10), which takes a random seed, $\texttt{seed}_{\texttt{tree}}$ and a salt, `salt` as inputs and outputs t seeds of the ephemeral matrices, $(\texttt{eseed}_0, ..., \texttt{eseed}_{t-1}) \in \{0, 1\}^{t\lambda}$. The seed generation process starts from the node of the 0-th `level` (i.e., root node) and generates its two child nodes using a pseudorandom function `XOF`. The same process continues for all nodes at each level, except for leaf nodes. After that, `SeedLeaves` algorithm (presented in Appendix D, Alg. 11) is used to store the leaf nodes ($\texttt{eseed}_0, ..., \texttt{eseed}_{t-1}$) of the GGM tree. This algorithm first stores the leaf nodes of the highest level, and then repeats the same process for the previous level that contains the leaf nodes. These ephemeral seeds are used in signature generation. Next, each ephemeral monomial matrix $\widetilde{\boldsymbol{Q}}_i$ is generated using the corresponding seed $\texttt{eseed}_i$, `salt` and the sequence i (line: 11 of Alg. 1). After that `RREF` is used on the multiplication of the public matrix $\boldsymbol{G}_0$ and the transpose of i-th ephemeral matrix $\widetilde{\boldsymbol{Q}}_i^T$ (i.e., $\boldsymbol{G}_0\widetilde{\boldsymbol{Q}}_i^T$). It computes another matrix $\widetilde{\boldsymbol{G}}_i$ and a pivot-column set say $\widetilde{J}$ such that the columns of $\boldsymbol{G}_0\widetilde{\boldsymbol{Q}}_i^T$ indexes by $\widetilde{J}$ are the pivot columns and the remaining columns (i.e., indexes by $\widetilde{J}^c$) are the non-pivot columns (line: 12, Alg. 1).

After that, the permutation of $\widetilde{\boldsymbol{Q}}_i$ in the pivot columns is saved as π_i^* (line: 13, Alg. 1) and the non-pivot columns of $\widetilde{\boldsymbol{G}}_i$ are saved as $\widetilde{\boldsymbol{A}}_i$ (line: 14, Alg. 1). Next `BLIND` function takes a seed $\texttt{seed}_{\texttt{blind}}$ and matrix $\widetilde{\boldsymbol{A}}_i$ of order $k \times (n-k)$ as input and returns another matrix $\widetilde{\boldsymbol{B}}_i$ of order $k \times (n-k)$ such that $\widetilde{\boldsymbol{B}}_i = \boldsymbol{E}_r \widetilde{\boldsymbol{A}}_i \boldsymbol{E}_c$ satisfies, where $\boldsymbol{E}_r \in Mono^k(q)$, $\boldsymbol{E}_c \in Mono^{(n-k)}(q)$ are generated using the seed $\texttt{seed}_{\texttt{blind}}$. Then the canonical form (CF) of the matrix $\widetilde{\boldsymbol{B}}_i$ is computed, which is another matrix, say $\widetilde{\boldsymbol{C}}_i$. All the matrices in the canonical form $\widetilde{\boldsymbol{C}}_0, ..., \widetilde{\boldsymbol{C}}_{t-1}$ are used along with `salt` and the message `msg` to generate the commitment `cmt`. This commitment `cmt` is used to generate the fixed-length challenge (also known as digest vector) $\boldsymbol{b} = (\boldsymbol{b}[0], ..., \boldsymbol{b}[t-1])$. $\boldsymbol{b}$ is of fixed length t and fixed weight w and each component $\boldsymbol{b}[i] \in \mathbb{Z}_s$, $i \in \{0, ..., t-1\}$. Next, a binary vector $\boldsymbol{U}$ is generated from the vector $\boldsymbol{b}$ (lines: 19 to 21, Alg. 1). Depending on the binary vector $\boldsymbol{U}$ and the GGM tree $\texttt{seed}'$, the signature component `path` is generated using the function GGMpath. Also, depending on the digest vector $\boldsymbol{b}$, the response $\{\texttt{rsp}_i\}$ is generated. If $i \in \mathbb{Z}_t$ such that $\boldsymbol{b}[i] \neq 0$, then $\text{CosetRep}\big(\boldsymbol{Q}_{\boldsymbol{b}[i]}, \pi_i^*\big)$ is stored as $\texttt{rsp}_i$. Finally, Alg. 1 outputs the signature $\tau = \Big\{\texttt{cmt}, \texttt{salt}, \texttt{path}, \{\texttt{rsp}_i\}_{i \in \mathbb{Z}_w}\Big\}$ of the message `msg`.

We refer to LESS-v2 only as LESS throughout the remainder of the paper.

3 Our fault assisted forgery attack on LESS

3.1 Finding a secret equivalent component for forgery

To reduce storage, LESS only stores the master seed $\texttt{seed}_{\texttt{sk}}$ as the long-term signing key `sk` and utilizes it during the signature algorithm. This master seed is expanded to produce seeds of the $(s-1)$ long-term secret monomial matrices, $\boldsymbol{Q}_1$, ... $\boldsymbol{Q}_{s-1}$ (line: 2,4, 25 of Alg. 1).

Apart from the long-term secret key, the signature algorithm uses t seeds $\mathtt{eseed}_0$, ..., $\mathtt{eseed}_{t-1}$ as ephemeral secret. These seeds are expanded to generate the t ephemeral secret monomial matrices $\widetilde{\boldsymbol{Q}}_0$, ..., $\widetilde{\boldsymbol{Q}}_{t-1}$.

$\mathtt{eseed}_0$, ..., $\mathtt{eseed}_{t-1}$ are generated from the seed $\mathtt{seed}_{\mathtt{tree}}$ which was earlier generated from the same secret key $\mathtt{sk}$ and the random salt, $\mathtt{salt}$. Since the seeds of the secret ephemeral matrices also depend on the random salt, these secret ephemeral matrices ($\widetilde{\boldsymbol{Q}}_0$, ..., $\widetilde{\boldsymbol{Q}}_{t-1}$) behave like random monomial matrices. From the property of the $\mathtt{XOF}$, these matrices will be different in each signature generation.

Therefore, one can use random monomial matrices in place of ephemeral secret monomial matrices without affecting the verification process. Possession of $\mathtt{seed}_{\mathtt{sk}}$ enables a trivial signature forgery. However, since $\mathtt{seed}_{\mathtt{sk}}$ is used as an input of the $\mathtt{XOF}$ (here $\mathtt{Keccak}$) function, recovering $\mathtt{seed}_{\mathtt{sk}}$ from any of the following values, faulted or otherwise, is equivalent to inverting the SHA-3 hash function. However, if an attacker obtains an alternative secret key equivalent component that is sufficient to generate valid signatures, they can carry out a signature forgery attack.

We first explain some observations in Lemma 1, Lemma 2, and Lemma 3, and then use these observations to establish our claim of signature forgery through the equivalent secret key component in Theorem 1.

Lemma 1. *Let* $\boldsymbol{Q}_1 = (\boldsymbol{u}_1, \pi_1)$, ..., $\boldsymbol{Q}_{s-1} = (\boldsymbol{u}_{s-1}, \pi_{s-1})$ *be* $(s-1)$ *long-term secret monomial matrices in LESS signature that are generated from the secret key* $\mathtt{seed}_{\mathtt{sk}}$. *Then, having the above* $s-1$ *secret monomial matrices is sufficient to forge a signature.*

Proof. The long-term secret key $\mathtt{seed}_{\mathtt{sk}}$ is used to initialize a state $\mathtt{state}_{\mathtt{sk}}$ (line: 1, Alg. 1), seed $\mathtt{seed}_{\mathtt{sk}}$ which has been used multiple times in the LESS Signature Alg. 1. We will explain the use cases as follows:

- The state $\mathtt{state}_{\mathtt{sk}}$ is used to generate the seed of the public key $\mathtt{seed}_{\mathtt{pk}}$ (line: 2, Alg. 1). Since the public key already contains the public seed $\mathtt{seed}_{\mathtt{pk}}$, the attacker does not need to compute $\mathtt{seed}_{\mathtt{pk}}$ at the time of forgery.
- The secret state, $\mathtt{state}_{\mathtt{sk}}$ is then used to generate seeds ($\mathtt{seed}_1$, ..., $\mathtt{seed}_{s-1}$) (line: 4, Alg. 1) of secret monomial matrices $\boldsymbol{Q}_1$, ..., $\boldsymbol{Q}_{s-1}$ (line: 26, Alg. 1). So, if the attacker has the secret monomial matrices $\boldsymbol{Q}_1$, ..., $\boldsymbol{Q}_{s-1}$, then the attacker can avoid computing these matrices. Lastly, the secret state, $\mathtt{state}_{\mathtt{sk}}$ is used to generate the blind state $\mathtt{state}_{\mathtt{blind}}$ (line: 9, Alg. 1). This blind state is used to mask the matrix $\boldsymbol{A}_i$ using the function $\mathtt{BLIND}$. This function takes $\mathtt{state}_{\mathtt{blind}}$ and the matrix $\boldsymbol{A}_i$ as input and returns another matrix $\boldsymbol{B}_i$ such that the following conditions are satisfied.
$$\boldsymbol{B}_i = \boldsymbol{E}_r \cdot \boldsymbol{A}_i \cdot \boldsymbol{E}_c,$$
where $\boldsymbol{E}_r \in Mono^k(q)$ and $\boldsymbol{E}_c \in Mono^{n-k}(q)$ are generated from the state $\mathtt{state}_{\mathtt{blind}}$. The output matrix $\boldsymbol{B}_i$ is used to generate $\boldsymbol{C}_i$ using the canonical function $\mathtt{CF}$.
Since the function $\mathtt{CF}$ has the following property
$$\mathtt{CF}(\boldsymbol{E}_r \cdot \boldsymbol{A}_i \cdot \boldsymbol{E}_c) = \mathtt{CF}(\boldsymbol{A}_i),$$
for any monomial matrices $\boldsymbol{E}_r \in Mono^k(q)$ and $\boldsymbol{E}_c^{n-k}(q)$. Therefore, bypassing the blind step does not hinder the verification process. Hence, we can skip the

blind state initialization process (line: 9, Alg. 1) and the masking process (line: 15, Alg. 1). This eliminates the requirement of the seed $\texttt{seed}_{\texttt{blind}}$, which relies on the secret key.

Since the above three steps, which depend on the secret key $\texttt{seed}_{\texttt{sk}}$, can be replaced by $s-1$ secret monomial matrices $\boldsymbol{Q}_1$, ..., $\boldsymbol{Q}_{s-1}$. Therefore, an adversary with the long-term secret monomial matrices $\boldsymbol{Q}_1$, ..., $\boldsymbol{Q}_{s-1}$ generates a valid signature with the same probability as an adversary or a legitimate signer with the long-term secret key $\texttt{seed}_{\texttt{sk}}$. □

Now we will describe a property of the `MonomialCompose` algorithm which has been used in the line: 27 of Alg. 1 in Lemma 2.

Algorithm 2 `MonomialCompose`

Input: A monomial matrix $\boldsymbol{Q} = (\boldsymbol{u}, \pi)$ and a partial permutation $\widetilde{\pi}^*$.
Output: The partial permutation $\pi^{*'} = \pi \circ \widetilde{\pi}^*$
1: **for** $i = 0$ to $K-1$ **do**
2: $\quad \pi^{*'}(i) = \pi(\widetilde{\pi}^*(i))$
3: Return $\pi^{*'}$

Algorithm 3 `MonoCom_New`

Input: A permutation π and a partial permutation $\widetilde{\pi}^*$.
Output: The partial permutation $\pi^{*'} = \pi \circ \widetilde{\pi}^*$
1: **for** $i = 0$ to $K-1$ **do**
2: $\quad \pi^{*'}(i) = \pi(\widetilde{\pi}^*(i))$
3: Return $\pi^{*'}$

Lemma 2. *Let* $(\boldsymbol{Q} = (\boldsymbol{u}, \pi), \widetilde{\pi}^*)$ *be a input to the* `MonomialCompose` *algorithm (Alg. 2), then the output of* `MonomialCompose` *algorithm is independent of the vector* $\boldsymbol{u}$.

Proof. Let $(\boldsymbol{Q}_1 = (\boldsymbol{u}_1, \pi), \widetilde{\pi}^*)$ and $(\boldsymbol{Q}_2 = (\boldsymbol{u}_2, \pi), \widetilde{\pi}^*)$ be two inputs of `MonomialCompose` algorithm (Alg. 2). Then, from the description of Alg. 2, it is easy to see that both inputs result in the same output of $\pi \circ \widetilde{\pi}^*$. □

Corollary 1. *The functions* `MonoCom_New` *in Alg. 3 and* `MonomialCompose` *in Alg. 2, produce the same output when they receive the inputs* $(\pi, \widetilde{\pi}^*)$ *and* $(\boldsymbol{Q} = (\boldsymbol{u}, \pi), \widetilde{\pi}^*)$, *respectively.*

The following lemma shows that if an adversary replaces Alg. 2 with Alg. 3 in Line: 26 of the signature generation algorithm (Alg. 1), then they will generate the same (valid) signature.

Lemma 3. *Let* $\boldsymbol{Q}_1 = (\boldsymbol{u}_1, \pi_1)$, ..., $\boldsymbol{Q}_{s-1} = (\boldsymbol{u}_{s-1}, \pi_{s-1})$ *be* $(s-1)$ *long-term secret monomial matrices in LESS signature. Then if the adversary only use the permutations* $\pi_1, ..., \pi_{s-1}$ *instead of the monomial matrices* $\boldsymbol{Q}_1, ..., \boldsymbol{Q}_{s-1}$ *during the signature generation, the adversary-generated signature will pass the verification with the same probability.*

Proof. Let $\widetilde{\boldsymbol{Q}}_0 = (\boldsymbol{v}_0, \widetilde{\pi}_0)$, ..., $\widetilde{\boldsymbol{Q}}_{t-1} = (\boldsymbol{v}_{t-1}, \widetilde{\pi}_{t-1})$ be the ephemeral matrices. $\widetilde{\pi}_0^*$, ..., $\widetilde{\pi}_{t-1}^*$ are partial permutation generated from corresponding permutation $\widetilde{\pi}_i$, ..., $\widetilde{\pi}_{t-1}$ as shown in line:13 of Alg. 2.1.

During the signature generation procedure, the long-term secret monomial matrices, $\boldsymbol{Q}_1$, ..., $\boldsymbol{Q}_{s-1}$, are used only in `MonomialCompose` function. This function is applied when the challenge $\boldsymbol{b}[i] \neq 0$, for $i \in \mathbb{Z}_t$ and $1 \leq [i] \leq s-1$.

From Corollary 1, `MonoCom_New` and `MonomialCompose` produce the same output given the following inputs: $(\pi_{\boldsymbol{b}[i]}, \widetilde{\pi}_i^*)$ and $(\boldsymbol{Q}_{\boldsymbol{b}[i]}=(\boldsymbol{u}_{\boldsymbol{b}[i]}, \pi_{\boldsymbol{b}[i]}), \widetilde{\pi}_i^*)$, for $i \in \mathbb{Z}_t$.

Therefore instead of using the long-term secret monomial matrices $\boldsymbol{Q}_1$, ... $\boldsymbol{Q}_{s-1}$, if adversary uses only their permutations π_1, ..., π_{s-1} with the new function `MonoCom_New`, then the output signature of the signature generation algorithm will not change. Consequently, the generated signature will pass the verification process with the same probability as before. □

Theorem 1. *Let $\boldsymbol{Q}_1=(\boldsymbol{u}_1, \pi_1)$, ..., $\boldsymbol{Q}_{s-1}=(\boldsymbol{u}_{s-1}, \pi_{s-1})$ be $(s-1)$ long-term secret monomial matrices in the LESS signature algorithm that are generated from the secret key $\mathtt{seed}_{\mathtt{sk}}$. To forge the signature, it is sufficient to have all the above $s-1$ permutations π_1, ..., π_{s-1} in place of having the secret key $\mathtt{seed}_{\mathtt{sk}}$.*

Proof. This theorem follows directly from Lemma 1 and Lemma 3. □

The complete signature forgery method is given in Appendix 12, where we use the permutations π_1, ..., π_{s-1} of secret monomial matrices as input instead of the secret key $\mathtt{seed}_{\mathtt{sk}}$.

3.2 Extraction of the secret equivalent permutation

From the previous section, we obtained that if an adversary acquires all the $s-1$ permutations $\pi_1, \cdots, \pi_{s-1}$ of the long-term secret monomial matrices, then they can forge signatures. In the following, we explain how an adversary can obtain the above $s-1$ permutations.

Suppose $\tau = \left\{\mathtt{cmt}, \mathtt{salt}, \mathtt{path}, \{\mathtt{rsp}_i\}_{i\in\mathbb{Z}_w}\right\}$ is a valid signature and $\boldsymbol{b}$ is the output of $\mathtt{SampleChallenge}(\mathtt{cmt})$ is the fixed challenge corresponding to the signature τ. Then, for each value $\boldsymbol{b}[i]$, either $\mathtt{eseed}_i$ (can be computed from `path`) or $\mathtt{CosetRep}\big(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\big)$ will be revealed, where $0 \le i \le t-1$. The seed $\mathtt{eseed}_i$ is also used to generate the corresponding monomial matrix $\widetilde{\boldsymbol{Q}}_i$ and $\mathtt{CosetRep}(\mu)$ outputs the set $\{\mu(i) : i \in [k]\}$, where μ is a partial permutation.

According to the mathematical security of LESS [4], an adversary cannot calculate the corresponding secret permutation $\pi_{\boldsymbol{b}[i]}$ from only one of the following two: (i) $\widetilde{\boldsymbol{Q}}_i$ and (ii) $\mathtt{CosetRep}\big(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\big)$, for any $0 \le i \le t-1$. However, in the following theorem, we show that if the adversary manages to obtain both: (i) $\widetilde{\boldsymbol{Q}}_i$ and (ii) $\mathtt{CosetRep}\big(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\big)$, corresponding to a non-zero value $\boldsymbol{b}[i]$ (since, $\mathrm{CosetRep}\big(\boldsymbol{Q}_{\boldsymbol{b}[i]}, \pi_i^*\big)$ is stored as $\mathtt{rsp}_i$, only for $\boldsymbol{b}[i] \neq 0$), they can obtain information regarding to the secret permutation $\pi_{\boldsymbol{b}[i]}$.

Theorem 2. *Let $\boldsymbol{b}[i] \in [1, s-1]$ be the r-th non-zero digest value and $\left(\widetilde{\boldsymbol{G}}_i, \widetilde{J}\right) = \mathtt{RREF}(\boldsymbol{G}_0 \widetilde{\boldsymbol{Q}}_i^T)$, where $0 \le i \le t-1$. Suppose we have both: (i) the seed $\mathtt{eseed}_i$ of the monomial matrix $\widetilde{\boldsymbol{Q}}_i = (\boldsymbol{v}_i, \widetilde{\pi}_i)$ and (ii) $\mathtt{rsp}_r = \mathtt{CosetRep}\big(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\big)$, where $\pi_{\boldsymbol{b}[i]}$ is the permutation of the long term secret matrix $\boldsymbol{Q}_{\boldsymbol{b}[i]}$ and $\widetilde{\pi}_i^* = \big(\widetilde{\pi}_i^{-1}\big)_{|\widetilde{J}}$. Then, we can recover the permutation $\pi_{\boldsymbol{b}[i]}$.*

Proof. In LESS Signature algorithm, $\widetilde{\boldsymbol{B}}_i = \mathtt{CF}(\widetilde{\boldsymbol{A}}_i)$ (line: 15 of Alg. 1) and in LESS Verify, $\widehat{\boldsymbol{B}}_i = \mathtt{CF}(\widehat{\boldsymbol{A}}_i)$ (line: 12 of Alg. 9). From the correctness of LESS, we have $\widetilde{\boldsymbol{B}}_i = \widehat{\boldsymbol{B}}_i$.

From our initial assumption, we know $\widetilde{\boldsymbol{Q}}_i$. Since, $\boldsymbol{G}_0$ is public, we can compute $\widetilde{\boldsymbol{G}}_i$, $\widetilde{J}$ and $\widetilde{\boldsymbol{A}}_i$ following line: 11-14 of Alg. 1 which satisfies the following conditions.

$$\left(\widetilde{\boldsymbol{G}}_i,\, \widetilde{J}\right) = \texttt{RREF}\left(\boldsymbol{G}_0\widetilde{\boldsymbol{Q}}_i^T\right),\ \widetilde{\boldsymbol{A}}_i = \widetilde{\boldsymbol{G}}_i[*,\, \widetilde{J}^c], \tag{1}$$

$$\left(\boldsymbol{I}_k \mid \widetilde{\boldsymbol{A}}_i\right) = \widetilde{\boldsymbol{S}}\boldsymbol{G}_0\widetilde{\boldsymbol{Q}}_i^T\widetilde{\boldsymbol{P}}, \tag{2}$$

where $\widetilde{\boldsymbol{S}}$ is an invertible matrix and $\widetilde{\boldsymbol{P}}$ is a permutation matrix of order n. Since the matrices $\boldsymbol{G}_0$ and $\widetilde{\boldsymbol{Q}}_i$ are known, we can easily compute $\widetilde{\boldsymbol{P}}$ and $\widetilde{\boldsymbol{S}}$.

We also assume that $\texttt{rsp}_r$ is known. Therefore, two sub-matrices $\boldsymbol{G}'_1$ and $\boldsymbol{G}'_2$ can be generated from $\boldsymbol{G}_{\boldsymbol{b}[i]}$ and the indexes $\texttt{rsp}_r$ and $\texttt{rsp}_r^c$, respectively (Line: 16 & 17 of Alg. 9). Therefore, the matrix $(\boldsymbol{G}'_1 \mid \boldsymbol{G}'_2)$ contains all the columns of $\boldsymbol{G}_{\boldsymbol{b}[i]}$ but in different order. So, there exists a permutation matrix, say $\widehat{\boldsymbol{P}}$, such that $\boldsymbol{G}_{\boldsymbol{b}[i]} = (\boldsymbol{G}'_1 \mid \boldsymbol{G}'_2)\widehat{\boldsymbol{P}}$ holds. This implies

$$\boldsymbol{G}'^{-1}_1 \cdot \boldsymbol{G}_{\boldsymbol{b}[i]} \cdot \widehat{\boldsymbol{P}}^{-1} = \left(\boldsymbol{I}_k \mid \boldsymbol{G}'^{-1}_1\boldsymbol{G}'_2\right) = \left(\boldsymbol{I}_k \mid \widehat{\boldsymbol{A}}_i\right) \tag{3}$$

As we know the $\boldsymbol{G}'_1$, $\boldsymbol{G}'_2$ and $\boldsymbol{G}_{\boldsymbol{b}[i]}$ matrices, we can compute the matrix $\widehat{\boldsymbol{P}}$ and $\widehat{\boldsymbol{A}}_i$. Since $\widetilde{\boldsymbol{A}}_i$ and $\widehat{\boldsymbol{A}}_i$ are known, therefore, we can find four monomial matrices $\widetilde{\boldsymbol{Q}}_r$, $\widehat{\boldsymbol{Q}}_r \in Mono^k(q)$ and $\widetilde{\boldsymbol{Q}}_c$, $\widehat{\boldsymbol{Q}}_c \in Mono^{n-k}(q)$ such that the following conditions hold.

$$\widetilde{\boldsymbol{B}}_i = \widetilde{\boldsymbol{Q}}_r\widetilde{\boldsymbol{A}}_i\widetilde{\boldsymbol{Q}}_c \text{ (since } \widetilde{\boldsymbol{B}}_i = \texttt{CF}(\widetilde{\boldsymbol{A}}_i)) \text{ and } \widehat{\boldsymbol{B}}_i = \widehat{\boldsymbol{Q}}_r\widehat{\boldsymbol{A}}_i\widehat{\boldsymbol{Q}}_c \text{ (since } \widehat{\boldsymbol{B}}_i = \texttt{CF}(\widehat{\boldsymbol{A}}_i)\ ) \tag{4}$$

From Equation 4, we have the following:

$$\widetilde{\boldsymbol{A}}_i = \boldsymbol{Q}'^{-1}_r\widehat{\boldsymbol{A}}_i\boldsymbol{Q}'_c, \text{ where } \boldsymbol{Q}'^{-1}_r = \widetilde{\boldsymbol{Q}}_r^{-1}\widehat{\boldsymbol{Q}}_r,\ \boldsymbol{Q}'_c = \widehat{\boldsymbol{Q}}_c\widetilde{\boldsymbol{Q}}_c^{-1} \tag{5}$$

Since we know all four matrices $\widetilde{\boldsymbol{Q}}_r$, $\widehat{\boldsymbol{Q}}_r$, $\widetilde{\boldsymbol{Q}}_r$ and $\widehat{\boldsymbol{Q}}_r$, we can compute $\boldsymbol{Q}'_r$ and $\boldsymbol{Q}'_c$ matrices. From Equation 2, we have

$$\begin{aligned}
\widetilde{\boldsymbol{S}}\boldsymbol{G}_0\widetilde{\boldsymbol{Q}}_i^T\widetilde{\boldsymbol{P}} &= \left(\boldsymbol{I}_k \mid \widetilde{\boldsymbol{A}}_i\right) = \left(\boldsymbol{I}_k \mid \boldsymbol{Q}'^{-1}_r\widehat{\boldsymbol{A}}_i\boldsymbol{Q}'_c\right), \text{ using Equation 5} \\
&= \boldsymbol{Q}'^{-1}_r\left(\boldsymbol{I}_k \mid \widehat{\boldsymbol{A}}_i\right)\begin{pmatrix}\boldsymbol{Q}'_r & 0\\ 0 & \boldsymbol{Q}'_c\end{pmatrix} = \boldsymbol{Q}'^{-1}_r\left(\boldsymbol{G}'^{-1}_1\boldsymbol{G}_{\boldsymbol{b}[i]}\widehat{\boldsymbol{P}}^{-1}\right)\begin{pmatrix}\boldsymbol{Q}'_r & 0\\ 0 & \boldsymbol{Q}'_c\end{pmatrix}, \text{ using Equation 3} \\
&\Longrightarrow \boldsymbol{G}_{\boldsymbol{b}[i]} = \left(\boldsymbol{G}'_1\boldsymbol{Q}'_r\widetilde{\boldsymbol{S}}\right)\boldsymbol{G}_0\left(\widetilde{\boldsymbol{Q}}_i^T\widetilde{\boldsymbol{P}}\begin{pmatrix}\boldsymbol{Q}'_r & 0\\ 0 & \boldsymbol{Q}'_c\end{pmatrix}^{-1}\widehat{\boldsymbol{P}}\right)
\end{aligned} \tag{6}$$

$\boldsymbol{G}_{\boldsymbol{b}[i]}$ and $\boldsymbol{G}_0$ are part of the public key that satisfies $\boldsymbol{G}_{\boldsymbol{b}[i]} = \boldsymbol{S}\boldsymbol{G}_0\left(\boldsymbol{Q}^{-1}_{\boldsymbol{b}[i]}\right)^T$. Therefore, from Equation 6, we can conclude that the permutation of the monomial matrix $\left(\boldsymbol{Q}^{-1}_{\boldsymbol{b}[i]}\right)^T$, $\pi_{\boldsymbol{b}[i]}$, is the same as the permutation of the monomial matrix $\widetilde{\boldsymbol{Q}}_i^T\widetilde{\boldsymbol{P}}\begin{pmatrix}\boldsymbol{Q}'_r & 0\\ 0 & \boldsymbol{Q}'_c\end{pmatrix}^{-1}\widehat{\boldsymbol{P}}$. Since the $\widetilde{\boldsymbol{Q}}_i^T$, $\widetilde{\boldsymbol{P}}$, $\boldsymbol{Q}'_r$, $\boldsymbol{Q}'_c$ and $\widehat{\boldsymbol{P}}$ matrices are known to us, we can easily compute the permutation $\pi_{\boldsymbol{b}[i]}$. □

4 Identification of attack surfaces

In Theorem 2, we have shown that for each non-zero digest value $\boldsymbol{b}[i]$, if both $\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\right)$ and the ephemeral matrix $\widetilde{\boldsymbol{Q}}_i$ are revealed then we can recover the

$\boldsymbol{b}[i]$-th secret permutation $\pi_{\boldsymbol{b}[i]}$. Note that revealing $\widetilde{\boldsymbol{Q}}_i$ is equivalent to revealing $\texttt{eseed}_i$. In a normal execution of a Zero-Knowledge identification protocol, this is not possible as it violates the witness-hiding property. Therefore, only one of the above responses is revealed at a time, i.e., when $\boldsymbol{b}[i] \neq 0$ we reveal $\texttt{CosetRep}\big(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\big)$, otherwise we reveal $\widetilde{\boldsymbol{Q}}_i$. Therefore, we resort to fault attacks to disrupt this execution flow and try to extract the secret permutations. Given the signature algorithm, it is not trivial to retrieve both responses. However, there are multiple design choices that are introduced to improve the efficiency of the signature algorithm or to reduce the signature size, which may make the implementation susceptible to fault attacks. Among possible attacks, signature forgery is the most detrimental for a signature algorithm, and achieving a forgery with a minimal number of fault injections is a non-trivial challenge.

In this work, we mainly focus on analyzing fault vulnerable locations after the commitment (cmt) generation. We will analyze each sequential step of response generation from the commitment. The execution flow is depicted in Fig. 1. We will

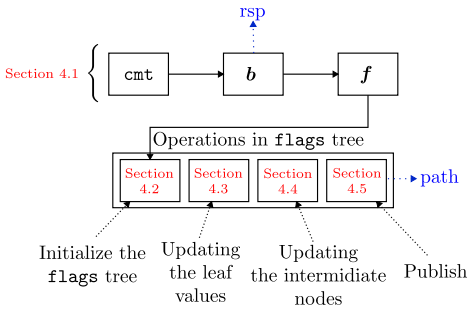


Fig. 1: Response generation (rsp, path) process from the commitment cmt and overview of this Sec. 4, where we discuss the possible attack surfaces.

briefly discuss whether each step in Fig. 1 is vulnerable to fault attacks or not. Note that we are only considering fault attacks that change/manipulate the value of a variable by using simple fault models such as instruction skip, stuck-at-one, or stuck-at-zero.

4.1 Changing cmt, digest vector $\boldsymbol{b}$ or the vector $\boldsymbol{f}$

Changing a value of cmt*:* We start by changing the value of the commitment cmt. From this cmt digest vector $\boldsymbol{b}$ is generated by the signer using the Fiat-Shamir transformation, in lieu of the random challenge from the verifier. Therefore, injecting fault on cmt will generate a random digest vector $\boldsymbol{b}'$, instead of $\boldsymbol{b}$. Therefore, the effect of the fault injection on cmt has the same effect as the fault injection on $\boldsymbol{b}$. In the following paragraph, we discuss the effect of fault injection on each step as drawn in Fig. 1 starting from $\boldsymbol{b}$.
Changing a value of $\boldsymbol{b}$*:* Suppose the i-th value of the digest vector $\boldsymbol{b}$ is non-zero in non-faulted case, which will lead the signer publishing $\texttt{CosetRep}(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*)$ as a response. However, we assume that we can change the value of $\boldsymbol{b}[i]$ to 0 by fault injection. Then, instead of publishing $\texttt{CosetRep}(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*)$, the signer will publish the information of $\texttt{eseed}_i$. Similarly, if we change the value of $\boldsymbol{b}[i]$ from 0 to any non-zero

value by injecting a fault, then instead of publishing the $\texttt{eseed}_i$, it will publish $\texttt{CosetRep}(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*)$. In these two cases, we cannot obtain both values. Therefore, this scenario is not suitable for us to compute the secret permutation $\pi_{\boldsymbol{b}[i]}$.

Changing a value of $\boldsymbol{f}$*:* $\boldsymbol{f}$ is a binary vector is generated from the digest $\boldsymbol{b}$ where

$$\boldsymbol{f}[i] = \begin{cases} 1 & \text{if } \boldsymbol{b}[i] \neq 0 \\ 0 & \text{Otherwise} \end{cases}$$

for all $0 \leq i \leq t-1$. This binary vector $\boldsymbol{f}$ is used only once to assign values to leaf nodes during flag tree generation. In the paper [18] Huang et al. have shown that if a value $\boldsymbol{f}[i]$ can be changed from 1 to 0 by fault injection, then the leaf node of the flag tree will be changed, which will leak the secret information.

However, in our analysis, we found that this binary vector $\boldsymbol{f}$ is an extra operation. The vector $\boldsymbol{f}$ has been used only once in the assignment operation $\texttt{flags}[\texttt{lsi}[i]+j] \leftarrow \boldsymbol{f}[k]$ (line: in Alg. 6). So, if we replace this operation with

$$\texttt{flags}[\texttt{lsi}[i]+j] = \begin{cases} 1 & \text{if } \boldsymbol{b}[k] \neq 0 \\ 0 & \text{Otherwise} \end{cases}$$

then we do not require the vector $\boldsymbol{f}$. Therefore, this vulnerability can be mitigated by eliminating the redundant binary vector generation.

As shown in Fig. 1, the next step to generate the response is `path` generation, which includes four steps. In this work, we mainly focus on fault attack analysis of these four steps, which include the binary incomplete `flags` tree generation and the publishing methods using this `flags` tree. The incomplete flag tree generation and publishing part from this tree divided into four parts. In this section, we will explain each part explicitly, along with the possible fault model and location corresponding to each part.

For non-zero digest value $\boldsymbol{b}[i]$, we will always get $\texttt{CosetRep}(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*)$. Therefore, to find our expected pair $\texttt{CosetRep}(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*)$ and $\texttt{eseed}_i$, we only need to recover the hidden seed $\texttt{eseed}_i$ corresponding to the non-zero value $\boldsymbol{b}[i]$ by fault injection. To explain the attack, we will explain the seed publish algorithm `GGMPath` in Alg. 4 that publishes the seed nodes of the GGM tree depending on the binary vector $\boldsymbol{f}$.

The GGMPath in Alg. 4 uses many arrays to track the index of a node in the incomplete GGM tree. We introduce them as follows: Let $T = \lceil \log(t) \rceil$. The given array $\texttt{n}\ell = (\texttt{n}\ell[0], \cdots, \texttt{n}\ell[T])$ defines the vector containing the number of nodes of the GGM tree, say $\texttt{seed}'$ per level. The arrays $\ell\ell = (\ell\ell[0], \cdots, \ell\ell[T])$ and $\texttt{off} = (\texttt{off}[0], \cdots, \texttt{off}[T])$ define the vectors containing the number of leaves of the GGM tree at each level and the vector containing the offsets that is required to represent the index of nodes per level, respectively. Also, let there be T' levels that contain a non-zero number of leaf nodes of the GGM tree. Let $\texttt{cons\_leaves} = (\texttt{cons\_leaves}[0], \cdots, \texttt{cons\_leaves}[T'-1])$ be the subvector of $\ell\ell$ containing all T' non-zero elements of $\ell\ell$ such that $t = \sum_{i=0}^{T'-1} \texttt{cons\_leaves}[i]$. Additionally, `lsi` is the vector containing the index of the starting leaf node for each of these T' levels.

We explain the four main steps in the response publishing process as follows:

Algorithm 4 `GGMPath`

Input: The seed tree $\texttt{seed}'$ and a binary vector $\boldsymbol{f}$.
Output: Returns the intermediate nodes of $\texttt{seed}'$ from which we can compute only the leaf nodes $\texttt{eseed}_i$ corresponding to $\boldsymbol{f}[i]=0$.

1: $\texttt{flags}[2t]=(1, 0 \cdots, 0)$ ▷ 1 represents the index of hidden seeds.
2: $\texttt{flags} \leftarrow \texttt{compute\_seeds\_to\_publish}(\texttt{flags}, \boldsymbol{f})$ ▷ Alg. 5
3: $\texttt{start\_node}=0$
4: **for** $\texttt{level}=1$ to T **do**
5: **for** $i=0$ to $\texttt{npl}[\texttt{level}]$ **do**
6: $\texttt{current\_node}=\texttt{start\_node}+i$
7: $\texttt{parent\_node}=\texttt{PARENT}(\texttt{current\_node})+\frac{\texttt{off}[\texttt{level}-1]}{2}$
8: **if** $\texttt{flags}[\texttt{current\_node}]=0$ and $\texttt{flags}[\texttt{parent\_node}]=1$ **then**
9: $\texttt{seed\_storage}[k] \leftarrow \texttt{seed}'_{\texttt{current\_node}}$
10: $k=k+1$
11: $\texttt{start\_node} \leftarrow \texttt{start\_node}+\texttt{npl}[\texttt{level}]$
12: Return $\texttt{seed\_storage}$

Algorithm 5 `compute_seeds_to_publish`

Input: A binary vector $\boldsymbol{f}=(\boldsymbol{f}[0], \cdots, \boldsymbol{f}[t-1])$ of size t and a binary tree $\texttt{flags}$ of size $2t-1$ that is initialized with all 0 except the first value $\texttt{flags}[0]$.
Output: Return the updated binary tree $\texttt{flags}$ that represents the index of the intermediate seeds of the GGM tree that should be published.

1: $\texttt{flags} \leftarrow \texttt{Label\_Leaves}(\texttt{flags}, \boldsymbol{f})$ ▷ Alg. 6
2: $\texttt{start\_node}=\texttt{leaves\_start\_indices}[0]$
3: **for** $\texttt{level}=T$ to 1 **do**
4: **for** $i=\texttt{n}\ell[\texttt{level}]-2$ to 0 **do**
5: $\texttt{current\_node}=\texttt{start\_node}+i$
6: $\texttt{parent\_node}=\texttt{PARENT}(\texttt{current\_node})+\frac{\texttt{off}[\texttt{level}-1]}{2}$
7: **if** $\texttt{flags}[\texttt{current\_node}]=0$ and $\texttt{flags}[\texttt{SIBLING}(\texttt{current\_node})]=0$ **then**
8: $\texttt{flags}[\texttt{parent\_node}]=0$
9: **else**
10: $\texttt{flags}[\texttt{parent\_node}]=1$
11: $i=i-2$
12: $\texttt{start\_node}=\texttt{start\_node}-\texttt{n}\ell[\texttt{level}-1]$
13: $\texttt{level}=\texttt{level}-1$
14: Return the binary tree $\texttt{flags}$

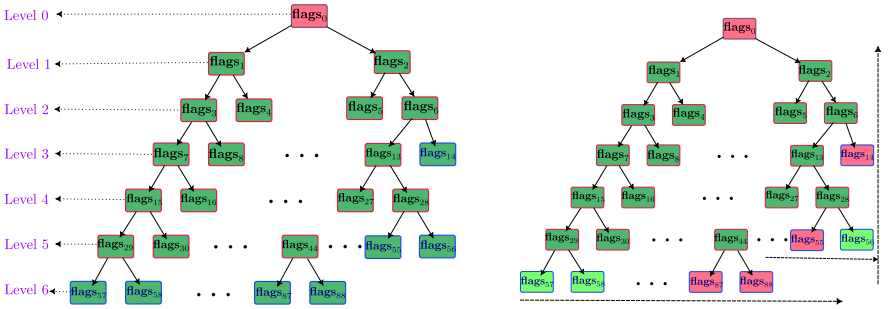
Algorithm 6 `Label_Leaves`

Input: A binary vector $\boldsymbol{f}=(\boldsymbol{f}[0], \cdots, \boldsymbol{f}[t-1])$ of size t and a binary tree $\texttt{flags}$ of size $2t-1$ that is initialized with all 0 except the first value $\texttt{flags}[0]$.
Output: Return the binary tree $\texttt{flags}$ that decides which nodes has to be published

1: $k \leftarrow 0$
2: **for** $i=0$ to T' **do**
3: **for** $j=0$ to $\texttt{cons\_leaves}[i]-1$ **do**
4: $\texttt{flags}[\texttt{lsi}[i]+j] \leftarrow \boldsymbol{f}[k]$
5: $k=k+1$
6: Return the binary tree $\texttt{flags}$

– ***Initializing the flag tree:*** According to the `GGMPath` in Alg. 4 (in line 1), it first initializes the flag tree by all 0 except the root. Here, 0 indicates the publish flag (dark green color node in Fig. 2a) and 1 means the hidden flag (red color node in Fig. 2a).
– ***Updating leaf nodes in the flag tree:*** Secondly, it updates each leaf node from dark green to red (if the corresponding value of $\boldsymbol{f}[i]=1$) or from dark green to light green (if the corresponding value of $\boldsymbol{f}[i]=0$, see Fig. 2b). This step overwrites each i-th leaf node values (initially all the values were 0) with the value $\boldsymbol{f}[i]$(line: 1- 5 in

Alg. 6). This overwriting process start from leftmost leaf node $\texttt{flags}[\texttt{lsi}[0]]$ at T-th level to overwrite it by $\boldsymbol{f}[0]$. After overwriting the node $\texttt{flags}[\texttt{lsi}[0]]$, the algorithm takes its immediate node on the right side and overwrites it with $\boldsymbol{f}[1]$. The same process continues until it completes overwriting all the T-th level nodes. Then it proceeds to level $T-1$ and repeats the process for all the leaf nodes of the other levels. In Fig. 2b, the directions $\rightarrow$ and $\uparrow$ indicate the sequence of accessing the leaf nodes.



(a) Flag tree initialization (b) Update in leaf nodes

Fig. 2: The flag tree generation process from a binary vector $\boldsymbol{f}$ (for LESS-252-45)

– ***Updating intermediate nodes in flag tree:*** Thirdly, the updation process of the parent node of each node, depending on the flag values of the node and its sibling node (see Fig. 3a). This process is described in the `compute_seeds_to_publish` algorithm in Alg. 5 (line: 2- 13). In this step, Alg. 5 first takes the second rightmost leaf node $\texttt{flags}[\texttt{leaves\_start\_indices}[0]+\mathrm{n}\ell[T]-2]$ node in the T-th level and its sibling node to update their parent node. After updating their parent, the algorithm iterates through their immediate leftmost child nodes, updating each in turn. The same process continued until it completed all parent-node updates for all the T-th level nodes. Then it proceeds to level $T-1$ and continues the same process up to level 1. In Fig. 3a, the directions $\leftarrow$ and $\uparrow$ indicate the sequence of accessing the nodes.

– ***The publishing part:*** Finally, the seed publishing process (line : 3- 11 in Alg . 4) is made using the complete flag tree. The checking process starts from the first flag node $\texttt{flags}[1]$ and its parent node $\texttt{flags}[0]$, and makes the decision. This process make ready the seed node $\texttt{seed}'_1$ to publish only if the corresponding node $\texttt{flags}[1]=0$ and $\texttt{flags}[0]=1$. Next, it goes to the immediate right node $\texttt{flags}[2]$ to perform a similar process i.e., check the condition $\texttt{flags}[2]=0$ and $\texttt{flags}[1]=1$ for decision making. If no node is left in the immediate rightmost position, it will move to the next level and repeat the process. In Fig. 3a, the directions $\leftarrow$ and $\downarrow$ indicate the sequence of accessing the nodes for decision making.

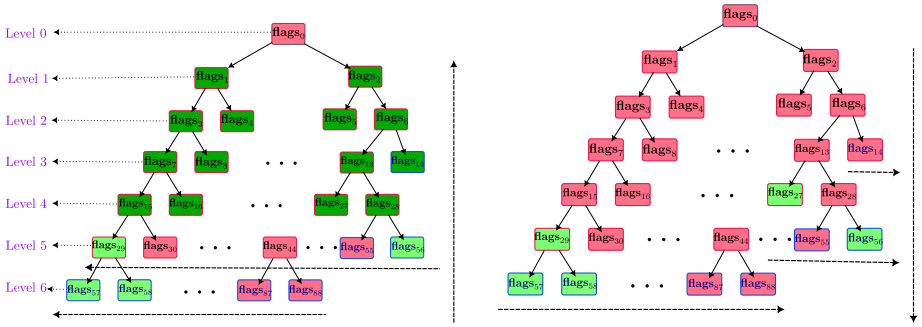
Finally, the dark blue seed nodes in Fig. 4 will be published. The remaining seed nodes that do not satisfy the publishing criteria can be divided into two categories:

– First category of non-publishing seeds: the seed node $\texttt{seed}'[i]$ falls in this category whose corresponding flag value $\texttt{flags}'[i]=0$ (green node) and the flag value of its parent is also 0 (green node). These types of seeds must lie in the sub-tree

rooted at a dark blue node. According to the GGM tree construction, these types of seeds can be generated from the blue seed node. These nodes are not hidden. For example: according to Fig. 4 the unpublished seed nodes $\texttt{seed}'_{57}$ and $\texttt{seed}'_{58}$ can be generated from their published parent $\texttt{seed}'_{29}$.

– The second category of non-publishing seeds: the seed node $\texttt{seed}'[i]$ has flag value $\texttt{flags}'[i]=1$ (red node). In this case, we cannot generate the seed $\texttt{seed}'[i]$ from the published blue seed nodes. Therefore, these types of seeds will be hidden.

In this work, our target is to trick the signature generation process to publish a second category non-publishing seed, say $\texttt{seed}'[i]$, corresponding to a red colored node $\texttt{flags}[i]$ (i.e., $\texttt{flags}[i]=1$).



(a) Update process of the complete flag tree (b) Flag tree checking step

Fig. 3: The flag tree generation process from a binary vector $\boldsymbol{f}$ (for LESS-252-45)

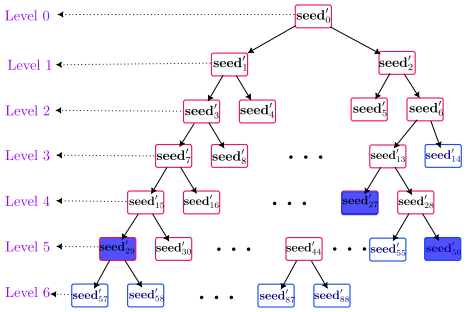


Fig. 4: Dark blue nodes represent the seeds to be published; others must be hidden

Attack Idea: Let $\texttt{flags}[i_l]$ be the i-th leaf node at the l-th level and $\texttt{flags}[i_{l-1}]$, ⋯, $\texttt{flags}[i_0]$ be the ancestors of $\texttt{flags}[i_l]$ in non-faulted cases, where $i_{j-1}=\texttt{PARENT}(i_j)$, and $1\leq j\leq l\leq T$. Suppose in the normal execution, the value of the node $\texttt{flags}[i_l]$ should be 1. Then, all the ancestors of $\texttt{flags}[i_l]$ should be 1. Therefore, in the publishing part (line: 3-11 in Alg. 4), none of the seeds $\texttt{eseed}_i=\texttt{seed}'_{i_l}$, ⋯, $\texttt{seed}'_{i_0}$ should be published in the non-faulted case. Let after all three node update steps are done, one of the nodes $\texttt{flags}[i_l]$, ⋯, $\texttt{flags}[i_0]$ say $\texttt{flags}[i_j]$ changed to 0 by a fault injection, i.e., $\texttt{flags}'[i_j]=0$, where $\texttt{flags}'$ is the faulted flag tree and $0\leq j\leq l\leq T$.

Then, $\texttt{flags}'[i_j] \neq \texttt{flags}[i_j]$ which affects its ancestor node generation. In this case, from the publishing part (line: 3-11 in Alg. 4), either $\texttt{seed}'_{i_j}$ or its one ancestor will be published where $0 \leq j \leq l \leq T$. Therefore, it will publish either the i-th leaf node $\texttt{seed}'_{i_l}$ (which is equal to $\texttt{eseed}'_i$) or its some ancestor $\texttt{seed}'_{i_j}$. In this scenario, the attacker either gets this hidden seed node $\texttt{eseed}_i$ directly or can compute $\texttt{seed}'_{i_l} (= \texttt{essed}_i)$, as the seeds are nodes of the GGM tree. The same effect will be observed if we change the value of any leaf or its ancestor node (i.e., any intermediate node). Additionally, if we can change multiple nodes from 1 to 0, we will obtain multiple hidden seed values. Our attack will work if the flag value of one or more nodes changes from 1 to 0 via single-fault injection. We assume the following statement:

Assumption 1: A fault can directly change the flag value of exactly one node $\texttt{flags}[i]$ (this can be any node) of the incomplete binary tree $\texttt{flags}$ from 1 to 0 or remains unchanged if the node is 0.

In the ZKFault paper [24], Mondal et al. presented a similar fault assumption for a complete binary tree in LESS-v1. As the complete binary tree construction was straightforward, in their case, a single fault can change at most one flag value directly from 1 to 0. Later, this faulted node updates only its ancestor nodes. Therefore, in this case, the flag values of the nodes that are neither directly faulted nor their ancestors will remain unchanged.

We applied this assumption to the newly introduced incomplete binary tree in LESS. To address this incomplete binary tree, the trace of indices of the incomplete flag tree is crucial. This index of the incomplete flag opens another attack surface for the fault attack. In this case, a single fault may change the values of multiple nodes at once. In this case, not only does the fault change a flag value from 1 to 0 or 0 to 0, it may change some values from 1 to 1 or 0 to 1. So, we introduce a new fault assumption, which is more generic.

Assumption 2: A single fault can directly change the flag values of multiple nodes of the incomplete binary tree $\texttt{flags}$ from $1 \rightarrow 0$ or $1 \rightarrow 1$ or $0 \rightarrow 1$ or $0 \rightarrow 0$.

Despite being a generic fault model, we can get the hidden seed information corresponding to multiple nodes that change from 1 to 0. Mostly based on these two fault assumptions, we explain each of the four steps of the $\texttt{flags}$ tree generation process in Sec . 4.2, Sec. 4.3, Sec. 4.4 and Sec. 4.5 and propose the possible fault attack models. As we use a single-fault model, the fault cannot be applied across multiple steps in flag tree generation. Therefore, whenever we do fault attack analysis in each of the four steps of flag tree generation, we will assume that the flag tree $\texttt{flags}$ has not been faulted in the previous steps. In all the sections, we use $\texttt{flags}$ as the flag tree and $\boldsymbol{f}$ as a binary tree for a challenge vector $\boldsymbol{b}$.

4.2 Initializing the flag tree

Suppose we skip the initialization of any node $\texttt{flags}[j]$ say, $j \in [0, 2t-1]$ (line: 1 of Alg. 4) in the flag tree by injecting a fault. Let's call this faulted flag tree as $\texttt{flags}'$. We discussed earlier that all leaf nodes are overwritten in the second step of leaf node update with the binary vector $\boldsymbol{f}$, and the intermediate nodes are updated in the third step. Therefore, in this case, the faulted tree $\texttt{flags}'$ will be the same as the non-faulted tree flags after the third step. Since any value in the flag tree is not changed by fault injection, we do not obtain our desired secret values by injecting a fault. So, this location is not suitable for fault attacks.

4.3 Updating leaf nodes in flag tree

The update part of leaf nodes of the flag tree follows the `Label_Leaves` algorithm (Alg. 6) using the binary vector $\boldsymbol{f}$. Three index trackers i, j, and k are used to sequentially overwrite the leaf nodes of the flag tree from the binary vector $\boldsymbol{f}$. So, if we change this sequence through a fault injection attack, the leaf-node update method of the flag tree will be interrupted. In this case, the `Label_Leaves` algorithm will overwrite one or many incorrect leaf node values. Since the leaf node will not be updated after this second step, the faulted leaf value will remain faulted after the intermediate node update step. According to the attack idea, we can obtain hidden seed information for all leaf nodes that change from 1 to 0 during fault injection.

Let, $l_0, \cdots, l_{T'-1}$ be T' levels that contains leaf nodes where $0 < l_{T'-1} \leq l_{T'-2} \leq l_0 = T$. And each l_i-th level contains $\texttt{cons\_leaves}[i]$ leaf nodes, and the index of starting leaf nodes is $\texttt{lsi}[i]$, where $0 \leq i \leq T'-1$. Here, we explain the possible attack strategies. For each strategy, we have explained several attack locations briefly as follows:

- **FIA-1:** [4] Skip first increment instruction $i = i+1$ of first for loop by injecting instruction skip fault: In this case, Alg. 6 will first update the leaf nodes of l_0-th level by $\boldsymbol{f}[0]$, $\boldsymbol{f}[1]$, $\cdots$ $\boldsymbol{f}[\texttt{cons\_leaves}[0]-1]$. Then we skip the increment instruction $i = i+1$ by instruction-skipping fault. So, in the next step, the value of i will be the same 0 instead of being 1. In this case, the leaf update will happen again for the l_0-th level rather than the l_1-th level with by

  $$\texttt{flags}'[\texttt{lsi}[0]+j] = \boldsymbol{f}[\texttt{cons\_leaves}[0]+j]$$

  Since $t < 2 \cdot \texttt{cons\_leaves}[0]$, for $0 \leq j < t - \texttt{cons\_leaves}[0]$ all nodes are updated with values from vector $\boldsymbol{f}$. After that, all the remaining nodes will be updated by garbage values as shown in Fig. 5. So, in this case, all leaf nodes are not overwritten sequentially by $\boldsymbol{f}$. In this case, each leaf node that takes the value from $\boldsymbol{f}$ will be changed from $0 \to 0$ or $1 \to 0$ or $0 \to 1$ or $1 \to 1$ by this instruction, skipping fault injection. This fault follows the fault assumption 2. Although we described the effect of skipping the first increment operation, a similar effect can be observed if we skip the other increment instruction in the first **for** loop.

[4] In Fig. 5, for both attack locations FIA-1 and FIA-2, the leaf nodes would try to be updated by reading some memory location that is not allocated to $\boldsymbol{f}$. Therefore, if there is any memory protection turned on in the system that prevents the program from doing such a read operation, then these fault attacks may result in a runtime error.

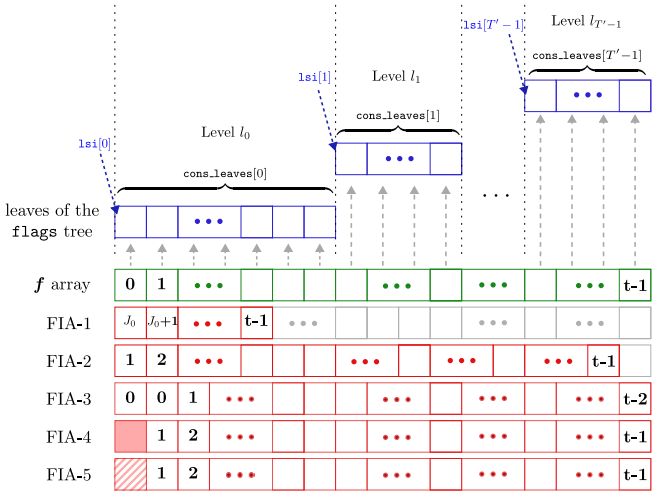


Fig. 5: The green array represents the indices of the **f** vector. The leaves of the `flags` tree are updated with the values at these indices as depicted in this figure. All the red arrays represent the vector of indices after the corresponding faults. In FIA-4 and FIA-5, the filled pattern represents the instruction skip fault at the first location, and the striped pattern represents the stuck-at-zero fault, respectively.

– **FIA-2:** Skip first increment instruction $j=j+1$ of second **for** loop in Alg. 6 by injecting instruction skipping fault:

  In this case, Alg. 6 will first update the leftmost leaf node of l_0-th level by $\boldsymbol{f}[0]$. Then we skip the increment instruction $j=j+1$. For $i=0$, the loop of j will take values like 0, 0, 1, 2, ⋯, `cons_leaves`$[0]-1$. In this case, the leaf update for the l_0-th level will be:

$$\texttt{flags}'[\texttt{lsi}[0]]=\boldsymbol{f}[0],\ \texttt{flags}'[\texttt{lsi}[0]]=\boldsymbol{f}[1],\ \texttt{flags}'[\texttt{lsi}[0]+1]=\boldsymbol{f}[2]\cdots$$
$$\texttt{flags}'[\texttt{lsi}[0]+\texttt{cons\_leaves}[0]-1]=\boldsymbol{f}[\texttt{cons\_leaves}[0]]$$

  In this case, all the leaf nodes of the flag tree will not be overwritten by the actual sequential value of $\boldsymbol{f}$. Similar to FIA-1, this fault model follows the fault assumption 2. However, if we skip the other increment instruction for the second loop, the same effect will be reflected.

– **FIA-3:** Skip first increment instruction $k=k+1$ inside the both for loops in `Label_Leaves` algorithm (Alg. 6) by injecting instruction skipping fault:

  In this case, Alg. 6 will first update the leaf most leaf node `flags'`$[\texttt{lsi}[0]]$ of l_0-th level by $\boldsymbol{f}[0]$. Then we skip the increment instruction $k=k+1$. So, in this case, for $i=0$ and $j=0, 1, \cdots,$ `cons_leaves`$[0]-1$, the k will be increment like 0, 0, 1, ⋯. So, the leaf updation for the l_0-th level will be:

$$\texttt{flags}'[\texttt{lsi}[0]]=\boldsymbol{f}[0],\ \texttt{flags}'[\texttt{lsi}[0]+1]=\boldsymbol{f}[0],\ \texttt{flags}'[\texttt{lsi}[0]+2]=\boldsymbol{f}[1]\cdots$$
$$\texttt{flags}'[\texttt{lsi}[0]+\texttt{cons\_leaves}[0]-1]=\boldsymbol{f}[\texttt{cons\_leaves}[0]-2]$$

In this case also, the leaf nodes of the flags tree at l_0-th level update with different values except the first leaf node $\texttt{flags}'[\texttt{lsi}[0]]$. Here, any leaf node (except the first one) can be set to 0 or 1. Therefore, this fault model follows the fault assumption 2.

- **FIA-4:** Here all the elements of flags are initialized by 0 except the root node. If we skip one assign instruction "$\texttt{flags}[\texttt{lsi}[0]+0] = \boldsymbol{f}[0]$" by instruction skipping fault injection, then the faulted value $\texttt{flags}'[\texttt{lsi}[0]+0]$ will be 0, as all the nodes of flag tree including the leaf node $\texttt{flags}[\texttt{lsi}[0]+0]$ is initialized by 0. All the remaining leaf nodes in the tree will remain unchanged. Since this fault can change only one node value directly, this model follows the fault assumption 1.
- **FIA-5:** There are other types of faults we could inject. Those are bit flip faults and stuck-at-zero faults. The bit flip fault or stuck-at-zero fault can change the value of only one node, say the first leaf node $\texttt{flags}[\texttt{lsi}[0]]$, from 1 to 0. This fault model follows the fault assumption 1.

4.4 Updating intermediate nodes in flag tree

The update part of intermediate nodes of the flag tree follows the algorithm `compute_seeds_to_publish` (Alg. 5). The flag tree update step takes a node and its sibling and, depending on the flag values of these nodes, updates the flag value of their parent. Alg. 5 first computes the index of a node and its parent by

$$\texttt{current\_node} = \texttt{start\_node} + i \;\&\; \texttt{parent\_node} = \texttt{PARENT}(\texttt{current\_node}) + \frac{\texttt{off}[\texttt{level}-1]}{2}$$

where `start_node`, i and `level` are variables. After this computation, the flag value of the parent node $\texttt{flags}[\texttt{parent\_node}]$ has been updated based on the flag value of `current_node` and its sibling. In this case, we will inject a fault in this execution such that the fault can generate (`current_node`, `parent_node`) index pair, i.e., the `parent_node` is not the actual parent node index of `current_node`. Then, the $\texttt{flags}[\texttt{current\_node}]$ will update the flag value of a node, say $\texttt{flags}[j]$, other than its actual parent node. So in this case, one fault injection may change both the values $\texttt{flags}[\texttt{parent\_node}]$ and $\texttt{flags}[j]$ which we wanted. We explained the possible attack location by skipping each assignment instruction corresponding to each of these variables `start_node`, `level`, `i`, `current_node`, and `parent_node` as follows:

- **FIA-6:** We skip the first instruction $\texttt{start\_node} = \texttt{start\_node} - \texttt{n}\ell[\texttt{level}-1]$ inside the loop for $\texttt{level} = T$ (line: 12 in Alg. 5) by injecting instruction skipping fault. This `start_node` represents the index of the leftmost node at the corresponding level. In the faulted case, the value of `start_node` takes incorrect value $\texttt{start\_node}' = \texttt{leaves\_start\_indices}[0]$ for $\texttt{level} = T-1$ as the correct one it this $\texttt{level} = T-1$ should be $\texttt{leaves\_start\_indices}[0] - \texttt{n}\ell[T-1]$. For this incorrect information of `start_node`, the algorithm takes a node $\texttt{current\_node} = \texttt{start\_node}' + i$, where i runs from $\texttt{n}\ell[T-1]-2$ to 0. So, each of these `current_node` lies in the T-th level but computation of its parent node will be according to the $T-1$-th level node, i.e., the index of the parent node of faulted node $\texttt{current\_node}'$ will be $\texttt{parent\_node}' = \texttt{PARENT}(\texttt{current\_node}') + \frac{\texttt{offset}[T-2]}{2}$. However, the actual parent node of this node is $\texttt{parent\_node} = \texttt{PARENT}(\texttt{current\_node}') + \frac{\texttt{offset}[T-1]}{2}$ as the node lies in the T-th level. In this

case, we can say that unless $\texttt{off}[T-1]=\texttt{off}[T-2]$, the $\texttt{current\_node}'$ node updates the flag value of $\texttt{flags}'[\texttt{parent\_node}']$ which is not the actual parent of $\texttt{current\_node}'$. Not only the faulted $\texttt{start\_node}$ In the $T-2$-th level, $\texttt{start}$ This fault follows the fault assumption 2, as it changes multiple flag values directly.

- **FIA-7:** Skip the decrement instruction $\texttt{level}=\texttt{level}-1$ inside the loop for $\texttt{level}=h$ (line: 13 in Alg. 5) by injecting instruction skipping fault, where $T\geq h\geq 1$: In this case, it will update the flag values of the parent nodes for all nodes from the T-th level to the h-th level. Then it updates the $\texttt{start\_node}$ as the leftmost node at $\texttt{level}=h-1$. Then, it skips the instruction $\texttt{level}=\texttt{level}-1$. So, in the next step, $\texttt{level}=h$ instead of $\texttt{level}=h-1$. In this case, the current node will run $\texttt{start\_node}+\texttt{n}\ell[h]-2$ to $\texttt{start\_node}$. Therefore, in this $\texttt{level}=h$, it will access all the $h-1$-th level's nodes, but it will compute their parent nodes considering them as h-level nodes. Therefore in the case, all the $\texttt{current\_node}$ and the corresponding $\texttt{parent\_node}$ pair will not be correct if $\texttt{off}[h-1]\neq\texttt{off}[h-2]$. The similar attack idea of **FIA-6** can be continued.
- Skip the decrement instruction $i=i-2$ inside the loop for $\texttt{level}=h$ and $i=i'$ (line: 11 in Alg. 5) by injecting instruction skipping fault, where $T\geq h\geq 1$ and $\texttt{n}\ell[h]-1\geq i'\geq 1$: For the fixed $\texttt{level}=h$, the value of faulted i will be $\texttt{n}\ell[h]-1$, $\texttt{n}\ell[h]-3$, $\cdots$, i', i'(for instruction skip), $i'-2$, $\cdots$,0 for fault injection, whereas in normal case the value of i will be $\texttt{n}\ell[h]-1$, $\texttt{n}\ell[h]-3$, $\cdots$, $i'+2$, i', $i'-2$, $\cdots$,0. Since the value of $\texttt{start\_node}$ and $\texttt{level}$ has not been changed by fault injection, the index of the $\texttt{current\_node}=\texttt{start\_node}+i$ and their corresponding parent of $\texttt{current\_node}$ will be the same as the non-faulted case for all $\texttt{n}\ell[h]\geq i\geq 1$. In this case, we cannot change the flag value of any intermediate node; therefore, we cannot obtain secret information.
- **FIA-8:** Skip the assignment instruction $\texttt{current\_node}=\texttt{current\_node}+i$ inside the loop for $\texttt{level}=h-1$ and $i=\texttt{n}\ell[h-1]-2$ (line: 5 in Alg. 5) by injecting instruction skipping fault, where $T\geq h\geq 1$: In this case, the faulted current node will be the leftmost current node in $\texttt{level}=h$ but it should be the rightmost node at $\texttt{level}=h-1$. So, following the same logic of FIA-6, the parent node computation will be faulty unless $\texttt{off}[h-1]=\texttt{off}[h-2]$.
- **FIA-9:** Skip the assignment instruction $\texttt{parent\_node}=\texttt{parent\_node}+\frac{\texttt{off}[h-1]}{2}$ inside the loop for $\texttt{level}=h$ and $i=i'$ (line: 6 in Alg. 5) by injecting instruction skipping fault, where $T\geq h\geq 1$ and $\texttt{n}\ell[h]-1\geq i\geq 0$: In this case, the faulted parent node will be the parent node of the previous current node, i.e., the index of faulted parent node will be $\texttt{parent\_node}'=\texttt{PARENT}(\texttt{current\_node}+2)+\frac{\texttt{off}[h-1]}{2}$ which is not equal to the actual parent node of the current node.
- **FIA-10:** We can change the value of one i-th node of the flag tree $\texttt{flags}'[i]$ from 1 to 0 by a bit flip or a stuck-at-zero fault. This fault will follow the fault assumption 1.

4.5 Publishing seed nodes part

The publishing part of intermediate seed nodes of the GGM tree using the flag tree follows the $\texttt{GGMPath}$ algorithm (Alg. 4). In this part, we assume that we can inject a fault such that the pair $\texttt{parent\_node}$ and $\texttt{current\_node}$ is not a correct parent-child

pair. Then, we cannot get extra hidden seed information as it checks the condition $\texttt{flags}[\texttt{current\_node}]=0$ and $\texttt{flags}[\texttt{parent\_node}]=1$. So, it will always hide the information for $\texttt{current\_node}=1$. Therefore, this location is useful for our attack.

In this section, we have explored the seed publishing process using the incomplete binary tree starting from the incomplete binary tree initialization. We have observed that there are 10 possible fault attack models that can reveal the hidden information we require. We can categorize these 10 fault models mainly into two parts:

- **Fault category 1:** We will say a fault model lies under the fault category 1 if the fault model follows the fault assumption 1. Among our described models, FIA-4, 5, and 10 lie in this category.
- **Fault category 2:** We will say a fault model lies under the fault category 2 if the fault model follows the fault assumption 2. Among the models we described FIA-1, 2, 3, 6, 7, 9, and 8 lie in this category.

5 Secret permutation recovery

As discussed earlier, if we have the pair $\left(\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[i]}\circ\widetilde{\pi}_i^*\right), \texttt{eseed}_i\right)$ for a non zero digest value $\boldsymbol{b}[i]$, then we can find the $\boldsymbol{b}[i]$-th permutation $\pi_{\boldsymbol{b}[i]}$ of long-term secret. Here we require all $s-1$ permutations $\left\{\pi_{\boldsymbol{b}[i]} : i\in[1,\, s-1]\right\}$ to forge a signature of a message. So, we need $s-1$ pairs related to the corresponding secret permutations $\{\pi_i :\ i\in[1,\, s-1]\}$. The number of recovered such pairs from a single faulted signature depends on the fault model and the fault attack location.

For example, if we use FIA-5 and change the 0-th leaf node $\texttt{flags}[\texttt{lsi}[0]]$ from 1 to 0 by bit flip fault, then the faulted signature contains only a single pair $\left(\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[0]}\circ\widetilde{\pi}_0^*\right), \texttt{eseed}_0\right)$. In this case, we will get only one permutation $\pi_{\boldsymbol{b}[0]}$ from one faulted signature. However, if we use the FIA-10 and change the 1-st node $\texttt{flags}[1]$ from 1 to 0 by a bit flip fault, then the faulted signature will contain the information of $\texttt{seed}'[1]$. From this seed, all the leaf seed nodes $\texttt{eseed}_0$, $\cdots$, $\texttt{eseed}_{l-1}$ at T'-th level can be generated, where $l=\texttt{cons\_leaves}[0]$. In this case, we may get multiple such pairs $\left\{\left(\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[i_j]}\circ\widetilde{\pi}_{i_j}^*\right), \texttt{eseed}_{i_j}\right) : 0\le i_j\le l-1 \ \wedge\ \boldsymbol{b}[i_j]\neq 0\right\}$ from one faulted signature. Therefore, in this case we can get multiple secret permutations $\left\{\pi_{\boldsymbol{b}[i_j]} :\ 0\le i_j\le l-1 \ \wedge\ \boldsymbol{b}[i_j]\neq 0\right\}$ from one faulted signature. The amount of secret information leakage depends on the fault model and its location.

Assuming a fixed fault model and location, we get r' pairs in $R=\left\{P_{i_j} : j\in\mathbb{Z}_{r'}\right\}$ from one faulted signature, where

$$P_{i_j}=\left(\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[i_j]}\circ\widetilde{\pi}_{i_j}^*\right), \texttt{eseed}_{i_j}\right).$$

Then, we will get r' secret permutations $\left\{\pi_{\boldsymbol{b}[i_j]} : \in\mathbb{Z}_{r'}\right\}$ where each permutation $\pi_{\boldsymbol{b}[i_j]}$ is computed from the corresponding pair P_{i_j}. So, two distinct pairs $P_{i_{j'}}$ and $P_{i_{j''}}$ can be used to compute the same secret permutation $\pi_{\boldsymbol{b}[i_{j'}]}$ if $\boldsymbol{b}[i_{j'}]=\boldsymbol{b}[i_{j''}]$. So, it is not sufficient to compute the number of pairs in R, instead we have to calculate the number of distinct non-zero digest values of the set $\left\{\boldsymbol{b}[i_j] :\ j\in\mathbb{Z}_{r'}\wedge P_{i_j}\in R\right\}$.

First, the question arises: if we obtain k faulted signatures under a fixed fault model and a location, then how many secret permutations can be recovered? We answer this

question in the next Section 5.1. The number of required faults will be the minimum value of k such that the number of recovered secret permutations from k faulted signatures will be approximately $s-1$, i.e., the total number of secret permutations.

In this paper, we call a fault effective, successful, or desired when we inject a fault that matches our targeted fixed fault model and location. Otherwise, we call it an ineffective fault. For example, suppose we assume to use FIA-5 and that the fault location is the 0-th leaf node $\texttt{flags}[\texttt{lsi}[0]]$ of the flag tree. Then we call a fault effective if it follows only the FIA-5 fault model and acts only on the 0-th leaf node. Otherwise, we will call it an ineffective fault. In this section 5.1, we will explain the amount of recovered secret permutations estimation process without doing the secret finding process. Although we have explained this calculation for only fault category 1. We also explained the hardness of this same calculation for category 2.

5.1 Information recovery from multiple faulted signature

Let $\texttt{Leaf\_Indices} = \{i_j : j \in \mathbb{Z}_r\}$ be the indices of leaf nodes, whose flag values could be affected by fault injection for a fixed fault model and location. The set $\texttt{Leaf\_Indices}$ is always constant for the fixed fault model. However, the corresponding values $\texttt{Leaf\_Flag} = (\texttt{flags}[i_j] : j \in \mathbb{Z}_r)$ of the $\texttt{Leaf\_Indices}$ depends on the corresponding digest values $\boldsymbol{b}$ which changes with signature.

Fault category 1: Under the fault category 1, exactly one node $\boldsymbol{flags'}[i]$ say will be changed from 1 to 0, or it remains 0 by fault injection. Therefore, under this fault category, the injected fault must affect only the leaf nodes rooted at $\texttt{seed}'[i]$. For each non-zero value of the mentioned affected leaf node, we must generate each pair P_{i_j} using the $\texttt{seed}'[i]$ and $\texttt{rsp}$. Therefore, we will get exactly r' pairs P_{i_j}, where r' is the number of ones of $\texttt{Leaf\_Flag}$.

The number of non-zero values of the vector $\texttt{Leaf\_Flag}$ changes with the signature. Assume that we have injected faults into k signatures and therefore we have k digest vectors $\boldsymbol{b}_i$ for $1 \le i \le k$. Corresponding to each $\boldsymbol{b}_i$, we get the vector $\texttt{Leaf\_Flag}_i$ for $1 \le i \le k$. Let W_i be the random variable that represents the number of non-zero elements of the vector $\texttt{Leaf\_Flag}_i$ for $1 \le i \le k$ and X be a random variable representing the number of distinct non-zero values of $\texttt{B} = \bigcup_{i=1}^{k} \{\boldsymbol{b}_i[i_j] : j \in \mathbb{Z}_r\}$. Then $0 \le W_i \le w$ and $0 \le X \le W_i \le s-1$, where w is the digest weight and the number of secret monomial matrices is $s-1$. The probability of getting m distinct secret permutations from k faulted signatures is given by:

$$\Pr[X=m] = \sum_{\ell_1=m}^{w} \sum_{\ell_2=m}^{w} \cdots \sum_{\ell_k=m}^{w} \Pr\left[X=m \;\middle|\; \bigwedge_{i=1}^{k} \{W_i=\ell_i\}\right] \cdot \Pr\left[\bigwedge_{i=1}^{k} \{W_i=\ell_i\}\right].$$

The total possible number of digest vectors of length t and weight w is $\binom{t}{w}(s-1)^w$. Also, the number of digest vectors that the corresponding digest sub-vectors that contain r leaf nodes in $\texttt{Leaf\_Flag}$ with weight ℓ is $\binom{r}{\ell}(s-1)^{\ell} \cdot \binom{t-r}{w-\ell}(s-1)^{w-\ell}$. Therefore

$$\Pr[W_i=\ell_i] = \frac{\binom{r}{\ell_i}(s-1)^{\ell_i} \cdot \binom{t-r}{w-\ell_i}(s-1)^{w-\ell_i}}{\binom{t}{w}(s-1)^w} = \frac{\binom{r}{\ell_i} \cdot \binom{t-r}{w-\ell_i}}{\binom{t}{w}}$$

Since all W_i's are independent and identically distributed, we have:

$$\Pr\left[\bigwedge_{i=1}^{k}\{W_i=\ell_i\}\right]=\prod_{i=1}^{k}\frac{\binom{r}{\ell_i}\cdot\binom{t-r}{w-\ell_i}}{\binom{t}{w}} \;\&\; \Pr\left[X=m \mid \bigwedge_{i=1}^{k}\{W_i=\ell_i\}\right]=\frac{m!\cdot\binom{s-1}{m}S(\ell,\, m)}{(s-1)^r}$$

where $\ell=\sum_{i=1}^{k}\ell_i$ and $S(\ell,\ m)$ represents the Stirling number of the second kind [27]. Therefore,

$$\mathbb{E}[X]=\sum_{m=1}^{s-1}m\cdot\Pr[X=m]=\sum_{m=1}^{s-1}m\cdot\sum_{\ell_1,\ell_2,\cdots,\ell_k}\left(\frac{m!\cdot\binom{s-1}{m}S(\ell,\, m)}{(s-1)^{\ell}}\cdot\prod_{i=1}^{k}\frac{\binom{r}{\ell_i}\cdot\binom{t-r}{w-\ell_i}}{\binom{t}{w}}\right).$$

This expected value $\mathbb{E}[X]$ represents the expected number of distinct non-zero digest values in the set $\mathtt{B}$ after k faults. For these $\mathbb{E}[X]$ distinct non-zero digest values, we will get $\mathbb{E}[X]$ secret permutations. This calculation only gives us the number of secrets we can recover with k faults under fault category 1.

Fault category 2: In fault category 2, we cannot always say that all the non-zero flag values in `Leaf_Flag` will be changed from 1 to 0. In this case, a non-zero value can also be overwritten by a non-zero value for fault injection. Therefore, we cannot calculate the number of recovered exact pairs P_{i_j} from the `Leaf_Flag`. It depends on the number of flag values in `Leaf_Flag` that change from 1 to 0, and the number of flag values in `Leaf_Flag` that change from 1 to 1. This calculation requires additional two probabilities $\Pr[\mathtt{flags}'[i_j]=1 \mid \mathtt{flags}[i_j]=1]$ and $\Pr[\mathtt{flags}'[i_j]=0 \mid \mathtt{flags}[i_j]=1]$ which must depends on the fault model and location. This calculation could be interesting, but for the sake of simplicity, we are excluding it from here. We provide the average number of recovered secret numbers by doing the simulations of each fault model and location in Section 5.2.

The following section explains the process of finding secret permutations from a faulted signature, along with the simulation results for each model and parameter set.

5.2 Secret recovery from a single fault

Suppose $\tau'=\left\{\mathtt{cmt},\, \mathtt{salt},\, \mathtt{path}',\, \{\mathtt{rsp}_i\}_{i\in\mathbb{Z}_w}\right\}$ is an effective faulted signature of a message `msg`. Let `flags` be the incomplete flag tree in the non-faulted case. This can be computed from the `cmt` of τ' as the commitment generation part is not faulted. Let `flags`$'$ be the flag tree that can be generated using the commitment `cmt` following the assumed fault model. Then, we can use the `RecoverSecret` algorithm (Alg. 7) to find the required secret permutations from one faulted signature τ'. Here, we use a binary array $\boldsymbol{a}$ of length $s-1$ (the total number of secret permutations) as a part of the input. The i-th element $\boldsymbol{a}[i]=0$ indicates that the i-th secret permutation π_i is still unknown, while $\boldsymbol{a}[i]=1$ indicates that the i-th secret permutation has already been recovered.

Simulation result corresponding to each models: We have calculated the average number of secret permutations recovered from the first-faulted signature across all models. For each fault model, we fixed a fault location and estimated the value of `count1` using Alg. 7 1000 times. We have provided the average simulation results for each case in Table 1, based on the following fault model and location, where each

Algorithm 7 `RecoverSecret`

Input: A faulted signature $\tau' = \left\{\texttt{cmt}, \texttt{salt}, \texttt{path}', \{\texttt{rsp}_i\}_{i\in\mathbb{Z}_w}\right\}$ corresponding to a message `msg`, public key $\texttt{pk} = (\texttt{seed}_{\texttt{pk}}, \boldsymbol{G}_1, \cdots, \boldsymbol{G}_{s-1})$ and a binary array $\boldsymbol{a}$ of size $s-1$.
Output: Return either some secret permutations $\{\pi_{\boldsymbol{b}[i]}\}$ of the secret monomial matrices $\boldsymbol{Q}_{\boldsymbol{b}[i]} = (\boldsymbol{f}_{\boldsymbol{b}[i]}, \pi_{\boldsymbol{b}[i]})$ and update the vector $\boldsymbol{a}$ or return $\perp$.

$(\boldsymbol{b}[0], \cdots, \boldsymbol{b}[t-1]) \leftarrow \texttt{SampleChallenge}(\texttt{cmt})$
for $i = 0$ to $t-1$ **do**
  if $\boldsymbol{b}[i] \neq 0$ **then** $\boldsymbol{f}[i] = 1$
  else $\boldsymbol{f}[i] = 0$
Compute $\texttt{flags} \leftarrow \texttt{compute\_seeds\_to\_publish}(\texttt{flags}, \boldsymbol{f})$
Compute $\texttt{flags}'$ using `cmt` according to the fault model.
$\texttt{count} \leftarrow 0$
for $i = 0$ to $2t-1$ **do**
  if $\texttt{flags}[i] = 1$ and $\texttt{flags}'[i] = 0$ **then**
    $\texttt{count} = \texttt{count} + 1$
if $\texttt{count} = 0$ **then return** $\perp$
$\texttt{count1} \leftarrow 0$
$\boldsymbol{G}_0 \leftarrow \texttt{SampleGenerator}(\texttt{seed}_{\texttt{pk}})$
$\texttt{eseed} = \left(\texttt{eseed}_{k_0}, \cdots, \texttt{eseed}_{k_{r'''-1}}\right) \leftarrow$ generated using $\texttt{path}'$ and `salt`
for $i = 0$; $i \leq r''' - 1$; $i = i+1$ **do**
  if $\boldsymbol{b}[k_i] \neq 0$ and $\boldsymbol{a}[k_i] = 0$ **then**
    $\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[k_i]} \circ \widetilde{\pi}^*_{k_i}\right) \leftarrow \{\texttt{rsp}_i\}_{i\in\mathbb{Z}_w}$
    Compute $\pi_{\boldsymbol{b}[k_i]}$ from $\texttt{CosetRep}\left(\pi_{\boldsymbol{b}[k_i]} \circ \widetilde{\pi}^*_{k_i}\right)$ and $\texttt{eseed}_{k_i}$ ▷ following Theorem 1
    $\boldsymbol{a}[k_i] = 1$, $\texttt{count1} = \texttt{count1} + 1$
if $\texttt{count1} = 0$ **then return** $\perp$
Return $\left\{\pi_{\boldsymbol{b}[k_i]} : \boldsymbol{b}[k_i] \neq 0 \text{ and } \boldsymbol{a}[k_i] \text{ is updated}\right\}$ and the updated vector $\boldsymbol{a}$

M-i: A, B, represents that the fault model is A and the fault location or type is B. For example, M-1 represents the fault model is FIA-1, and here we skip the first increment operator $i = i+1$. In both M-9A and M-9B, the fault model is the same, i.e., FIA-9, but the location is different. In FIA-9A, we skip the assign instruction $\texttt{parent\_node} = \texttt{PARENT}(\texttt{current\_node}) + \frac{\texttt{off}[\texttt{level}-1]}{2}$ when $\texttt{current\_node} = 3$. In FIA-9B, we skip the same instruction when $\texttt{current\_node} = 65$. In Table 1, we can see that in some cases the average number of recovered secret permutations equals the total number of secret permutations $(s-1)$; in the remaining cases, it does not. Therefore, we must continue the secret recovery process until the total number of recovered secret permutations reaches $s-1$. So, we need multiple faults in that case. The second row corresponding to each M-i, represents the average number of required faults for all $s-1$ secret permutation recovery.

6 Effective fault detection method

Let, $S = \{\pi_1, \cdots, \pi_{s-1}\}$ be $s-1$ secret permutations. So far, we have considered the secret permutation recovery process using the faulted signature that follows the targeted fault model (i.e., the effective faults). In this case, we always get the actual secret permutation S. However, obtaining the effective faulted signatures is not always possible in a noisy, practical setup. So, we may get the wrong secret permutations from an ineffective faulted signature. A trivial checking process of whether the recovered $s-1$ secret permutations $S' = \left\{\pi'_1, \cdots, \pi'_{s-1}\right\}$ is same with the actual secret permutation S can be done using following the steps:

Level		I			II		III	
Parameters		252-45	252-68	252-192	400-102	400-220	548-137	548-345
$s-1$		7	3	1	3	1	3	1
M-1	# Secrets per fault	5.826	2.994	1	1.241	0.999	1.503	1
	# Faults for full secret	2.014	1.008	1	4.076	1.001	3.063	1
M-2	# Secrets per fault	5.16	2.991	1	2.999	1	3	1
	# Faults for full secret	2.577	1.011	1	1.002	1	1	1
M-3	# Secrets per fault	4.963	2.995	1	2.999	1	3	1
	# Faults for full secret	2.695	1.009	1	1.001	1	1	1
M-4	# Secrets per fault	0.951	0.98	0.197	0.998	0.292	0.982	0.231
	# Faults for full secret	19.327	5.628	5.087	5.546	3.268	5.454	4.751
M-5	# Secrets per fault	0.951	0.98	0.197	0.998	0.292	0.982	0.231
	# Faults for full secret	19.327	5.628	5.087	5.546	3.268	5.454	4.751
M-6	# Secrets per fault	6.972	3	1	3	1	3	1
	# Faults for full secret	1.051	1	1	1	1	1	1
M-7	# Secrets per fault	0.004	3	0	3	1	3	1
	# Faults for full secret	1088	1	0	1	1	1	1
M-8	# Secrets per fault	1.457	2.539	0	1.086	0.514	1.699	0.384
	# Faults for full secret	12.462	1.51	0	4.863	1.97	2.756	2.64
M-9A	# Secrets per fault	6.856	3	1	3	1	3	1
	# Faults for full secret	1.175	1	1	1	1	1	1
M-9B	# Secrets per fault	1.463	1.826	0.968	1.665	0.997	2.86	1
	# Faults for full secret	12.128	2.591	1.044	2.928	1.001	1.165	1
M-10	# Secrets per fault	6.856	3	1	3	1	3	1
	# Faults for full secret	1.175	1	1	1	1	1	1

Table 1: Simulation results for all fault models related to parameter sets of LESS

- Generate one signatures σ_1 corresponding to a messages $\texttt{msg}_1$ using these recovered secret permutations S', public key $\texttt{pk}$ and the messages $\texttt{msg}_i$ in the `Less Signature Forge` Alg. 12.
- Verify the message signature pairs ($\texttt{msg}_1$, σ_1) using the `LESS Verify` Alg. 9. If the signatures cannot be verified successfully, then we can say that the recovered set of secret permutations S' is not the targeted secret permutations S. In this case, we will restart the secret permutation recovery process. If the signature verifies successfully, then we will generate another signature σ_2 following step 1, and do the rest.
- After a certain time, if we can always check if the signatures generated using computed secret permutations S' are successfully verified, then we can say that the recovered secret permutations set is our targeted secret permutation with high probability.For example: according to Fig. 4 the node $\texttt{seed}'_{29}$ will be published but its children $\texttt{seed}'_{57}$ and $\texttt{seed}'_{58}$ will not published. However, from the published node $\texttt{seed}'_{29}$, both the children can be generated, so they are not hidden nodes.

However, in this case, we first need to use one or more faulted signatures without knowing whether they are effective or ineffective. After using a certain number of faults, we can determine whether the recovered secret permutations are noisy. If we find that the recovered secrets are noisy, we have to recompute the secret permutations.

Therefore, after receiving the faulted signature, if we can immediately check whether it is an effective faulted signature, then we do not need to follow the above

noisy/actual secret permutations checking process further. Here, we are explaining the filtering method of the effective fault for the fault models under the fault category 1.

Fault detection method for the models under fault category 1: In this section, we explain the model M-5 which is under the fault category 1. Here, we use stuck-at-zero fault, $\mathtt{A}_0$: $\mathtt{flags}'[\mathtt{lsi}[0]]=0$. However, similar detection methods can also work for other fault models under the fault category 1.

Let we receive the faulted signature $\tau'=\left\{\mathtt{cmt},\ \mathtt{salt},\ \mathtt{path}',\ \{\mathtt{rsp}_i\}_{i\in\mathbb{Z}_w}\right\}$ by injecting a fault following the model M-5, where $\mathtt{path}'=\left(\mathtt{seed}'_{j_0},\ \cdots,\ \mathtt{seed}'_{j_{r'-1}}\right)$. Here, we will check whether the fault injection has been successful, i.e., we can generate the flag tree $\mathtt{flags}''$ from the commitment $\mathtt{cmt}$ by assigning the value $\mathtt{flags}''[\mathtt{lsi}[0]]=0$ in FIA-5. This $\mathtt{flags}''$ will be our expected faulted flag tree, as we expect the faulted signature to be generated in our assumed fault model. Let $\mathtt{flags}'$ be the actual flag tree generated in a practical fault environment, and the signature τ' is generated depending on the flag tree $\mathtt{flags}'$. We cannot see the actual faulted flag $\mathtt{flags}'$, but we can compute our expected flag tree $\mathtt{flags}''$ (that will look as if we assign the value $\mathtt{flags}'[\mathtt{lsi}[0]]=0$). If our guessed faulted flag tree $\mathtt{flags}''$ is equal to the actual one $\mathtt{flags}'$, then we will say that we correctly guessed our faulted location, i.e., the stuck-at-zero fault $\mathtt{A}_0$ has been successfully done. Otherwise, we can conclude that the locations are different. The guessed location has been checked, but the output faulted signature does not match it. Then, we will drop this signature, take another faulted signature, and repeat until we recover the complete set of secret permutations. Now, we will explain the complete checking process of the expected faulted $\mathtt{flags}''$ is equal to the practical faulted tree $\mathtt{flags}'$ as follows:

- **Step 1:** Depending on the fault model, we first compute the expected flag tree $\mathtt{flags}''$ from the commitment $\mathtt{cmt}$. Here, we expect the fault instruction to be $\mathtt{A}_0$.
- **Step 2:** Compute the indies say $\boldsymbol{E}_j := (j'_0,\ \cdots,\ j'_{r''-1})$ of the seed tree, those should be published according to our expected flag tree $\mathtt{flags}''$.
- **Step 3:** If the size of $\boldsymbol{E}_j$, r'' is not equal to the number of seeds published in the given faulted signature r', then clearly we will conclude that the expected flag tree $\mathtt{flags}''$ is not same with the practical faulted flag tree $\mathtt{flags}'$. So, we will guess another fault location and do the process from step 1.
- **Step 4:** If $r'=r''$, then we can put each seed value $\mathtt{seed}'_{j_x}$ of the given $\mathtt{path}'$ in the corresponding location j'_x, where $0\leq x\leq r''-1$. Then, the leaf nodes corresponding to each seed will be computed. Without loss of generality, say the computed leaf seed nodes are: $\mathtt{eseed}''=\left\{\mathtt{eseed}_{k_0},\ \cdots,\ \mathtt{eseed}_{k_{r'''-1}}\right\}$, where each $0\leq k_x\leq t-1$.
- **Step 5:** For each k_x location we will compute $\widehat{\boldsymbol{C}}_{k_x}$ by following the portion for $\boldsymbol{b}'[k_x]=0$ in verification algorithm, where $0\leq k_x\leq t-1$. For remaining location, we will follow the else part of computing $\widehat{\boldsymbol{C}}_{k_y}$, where $k_y\in\left\{j'_0,\ \cdots,\ j'_{r''-1}\right\}^c$ using $\{\mathtt{rsp}_i\}_{i\in\mathbb{Z}_w}$. Then recomputes the commitment $\mathtt{cmt}''$ using all of these $\widehat{\boldsymbol{C}}_0,\ \cdots,\ \widehat{\boldsymbol{C}}_{t-1}$ and given $\mathtt{salt}$ and $\mathtt{msg}$. If our guessed flag tree $\mathtt{flags}''$ is same with the practical faulted flag tree $\mathtt{flags}'$, then these all matrices $\widehat{\boldsymbol{C}}_0,\ \cdots,\ \widehat{\boldsymbol{C}}_{t-1}$ computed from the $\mathtt{eseed}''=\left\{\mathtt{eseed}_{k_0},\ \cdots,\ \mathtt{eseed}_{k_{r'''-1}}\right\}$ and $\{\mathtt{rsp}_i\}_{i\in\mathbb{Z}_w}$ will be

correct. So, in this case, it will create the same commitment $\texttt{cmt}''$ with the given commitment $\texttt{cmt}$. Otherwise, if any of the computed locations is false, then it will create a false leaf seed that will create wrong $\widehat{C}_i$. Therefore, in this case, the recomputed commitment will differ.

- **Step 6:** If $\texttt{cmt} = \texttt{cmt}''$, the guessed fault location is correct; otherwise, it is incorrect.

The fault detection method will help us leverage the successful fault signature and knowledge of the correct fault location. If we change the fault model, the fault detection model can be changed accordingly.

Fault detection method for the models under fault category 2: The similar fault detection process will not work for all the fault models under the fault category 2. For each fault model under this category, an effective fault can also changed a value $\texttt{flags}'[i]$ say from 0 to 1. In this case, we get neither the corresponding $\texttt{eseed}[i]$ nor the $\texttt{CosetRep}\big(\pi_{\boldsymbol{b}[i]} \circ \widetilde{\pi}_i^*\big)$ will be published. Therefore, we cannot verify **Step 5** although the fault is effective.

In this case, we will check up to **Step 3** similarly, which can discard some ineffective faulted signatures but not all. Then we compute the secret permutations using Theorem 2. After that, we will follow the trivial checking methods of whether the recovered $s-1$ secret permutations are the same as the actual secret permutation.

7 Practical Experiment

Each described fault model is based on one of the following: an instruction skip fault, a stuck-at-zero fault, or a bit-flip fault. All of these fault models are realistic and feasible. However, we experimentally validate FIA-9, a fault model under fault category 2 and FIA-10, a fault model under fault category 2 presented in Section 4.3 and 4.4 against the implementation of LESS parameter LESS-252-45.

Experimental Setup In the practical experiment, we use the ChipWhisperer platforms from NewAE Technology Inc. [25]. More specifically, the ChipWhisperer-Lite (CW-Lite) is used to generate clock glitches, and the CW308 board, together with a Cortex-M4 microcontroller containing an STM32F303 chip from STMicroelectronics, is used as the target device. A 7.37MHz clock signal is generated by CW-Lite and supplied to the target via the CW308. The host PC is connected via USB to the CW308 and controls the clock glitch parameters through a 20-pin connector.

We target the implementation of the signature generation of **LESS-252-45**, which is written in C, compiled with `gcc`, and converted into ARM assembly. After the execution of Alg. 5, the output flags (`flags`) are transmitted via USB. The Keysight oscilloscope DSOX3054A is used to measure execution cycles of each loop iteration of Alg. 5 when running on the Cortex-M4 board. It is done by inserting a trigger signal at the beginning of each loop iteration (line: 3) of Alg. 5. Note that it is only used during the evaluation phase to specify the clock cycles required for the process of LESS and synchronize the injection timing of clock glitches, as the

implementation is public. We follow the NewAE tutorial to induce a clock glitch. The glitch is configured with the following four parameters:

- **Width**: It denotes the width of the glitch pulse, which is fixed at 5 in this experiment.
- **Offset**: It denotes the percentage adjustment of the glitch initiation position, ranging from -40 to 40.
- **Delay**: It denotes the number of cycles from the trigger to the injection of the glitch. In this experiment, the Delay sweep starts from 30 clock cycles after the trigger in order to account for the latency between the beginning of the computation and the trigger signal. One loop iteration of Alg. 5 (line: 3) requires approximately 1326 clock cycles (about 180 μs), and this configuration ensures that a single glitch affects at most one update operation. To fully cover the execution time corresponding to the processing, the Delay was swept up to 1500 clock cycles with a resolution of 1 clock cycle.
- **Repeat**: It is the number of repetitions of the glitch injection, which is fixed to 1 in this experiment. This setting ensures that the clock glitch impacts only a single instruction of the target.

Experimental Results Fig. 6 and Fig. 7 show the experimental results of fault-occurrence distributions obtained by varying the glitch Delay. In these figures, the vertical axis represents the glitch Offset in Fig. 6, whereas it indicates the index of the output at which a fault appears in Fig. 7 when the Offset is fixed at 6 in Fig. 6. Due to the large number of sweep points of Delay, the results shown in Fig. 6 and Fig. 7 are aggregated over intervals of 10 clock cycles along the Delay axis. Each plotted point, therefore, represents the number of fault occurrences observed within the corresponding Delay interval. The color intensity of each point reflects the frequency of fault occurrence, where darker points indicate a higher number of observed faults. In Fig. 6, faults are observed only at specific Offset values. This is because only when the glitch is injected at particular Offsets does the clock period become sufficiently shortened for the CW308 board to recognize it as an invalid clock edge. Figure 7 illustrates, based on theoretical predictions of which outputs should be affected when a fault occurs at a specific internal state, which output indices exhibit faults at which Delay values. As a result, the observed distributions reflect the progression of the internal processing in Alg. 5 as a function of Delay.

***Fault Injection Attack-9**: Instruction Skip.* In this fault-injection attack, we target the following instruction skip when `current_node`$=65$ and 67.

`parent_node`$=$`PARENT(current_node)`$+$`(off[level`$-1]>>1$`);`

From a theoretical perspective, skipping the parent-node update operation is expected to cause a characteristic anomaly at the corresponding output index. Fig. 7a confirms this prediction by showing that faults occur at the expected output index only within a narrow Delay range corresponding to the execution cycles of the parent-node update.

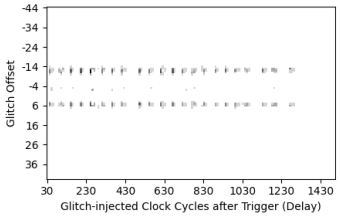


Fig. 6: Distribution of fault occurrences consistent with theoretical models.

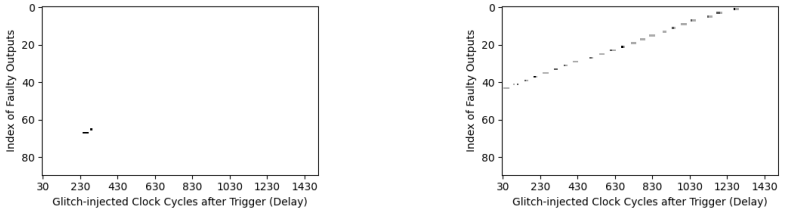


Fig. 7: Observed outputs under (a) Instruction skip fault & (b) Stuck-at-0 fault models.

Fault Injection Attack-10*: Stuck-at-0.* In this fault-injection attack, flags are observed after flipping from $1 \rightarrow 0$ in decreasing order on even indices (44,42,40,...) as the delay increases. It matches the structure of the GGM tree, where only even-indexed nodes are updated as current nodes, and odd indices (siblings) remain unchanged. As shown in Fig. 7b, these faults occur over a relatively wide Delay range and exhibit a clear periodic pattern. This periodic structure directly reflects the repeated execution of the update blocks in Alg. 5.

From these experimental results, we conclude that both the stuck-at-0 and instruction skip fault models reflect the multi-cycle execution structure of each loop iteration in Alg. 5. Consequently, even in the presence of noisy or spurious faults, an attacker can selectively filter out only those traces that are consistent with the theoretical fault models and perform a reliable analysis. Furthermore, in our experiments, the repeated observation of the same faults under identical glitch parameters confirms that the instruction skip fault model is realistically reproducible on actual hardware.

8 Conclusions

This work explores a fault attack analysis of the LESS signature algorithm, particularly the ephemeral seed publishing method that uses an incomplete decision tree. This publishing method and the incomplete decision tree are crucial because they are required to reduce the signature size of LESS. We propose multiple fault attack surface locations and attack models. We have discussed the desired fault detection method, which is essential to get an error-free secret component in a noisy environment. As the use of the GGM Tree and ZKP is relatively new in the construction of signature schemes, more such work is needed to further uncover vulnerable locations. This also highlights the need to design robust countermeasures to stop this type of attack. Additionally, our analysis uncovered many components of LESS design that can be removed entirely or replaced with simpler components. Removing these will not only thwart many potential physical attacks but also potentially improve speed and memory usage.

References

1. Abdalla, M., An, J.H., Bellare, M., Namprempre, C.: From Identification to Signatures via the Fiat-Shamir Transform: Minimizing Assumptions for Security and Forward-Security. In: Knudsen, L.R. (ed.) Advances in Cryptology - EUROCRYPT 2002, International Conference on the Theory and Applications of Cryptographic Techniques, Amsterdam, The Netherlands, April 28 - May 2, 2002, Proceedings. Lecture Notes in Computer Science, vol. 2332, pp. 418–433. Springer (2002). https://doi.org/10.1007/3-540-46035-7_28, https://doi.org/10.1007/3-540-46035-7_28
2. Adj, G., Aragon, N., Barbero, S., Bardet, M., Bellini, E., Bidoux, L., Chi-Domínguez, J., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M., Rivera-Zamarripa, L., Sanna, C., Tillich, J.P., Verbel, J., Zweydinger, F.: Mirath Signature Scheme (July 2025), https://pqc-mirath.org/assets/downloads/mirath_v2.0.1.pdf
3. Albrecht, M.R., Bernstein, D.J., Chou, T., Cid, C., Gilcher, J., Lange, T., Maram, V., von Maurich, I., Misoczki, R., Niederhagen, R., et al.: Classic mceliece. National Institute of Standards and Technology, Tech. Rep (2020)
4. Baldi, M., Barenghi, A., Beckwith, L., Biasse, J., Chou, T., Esser, A., Gaj, K., Karl, P., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.J.O., Santini, P., Wallace, R., Zweydinger, F.: LESS: Linear Equivalence Signature Scheme (March 2025), https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/less-spec-round2-web.pdf
5. Baldi, M., Barenghi, A., Beckwith, L., Biasse, J., Esser, A., Gaj, K., Mohajerani, K., Pelosi, G., Persichetti, E., Saarinen, M.J.O., Santini, P., Wallace, R.: Less: Linear equivalence signature scheme - Specification Document (Jan 2022), https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/less-spec-web.pdf
6. Baldi, M., Barenghi, A., Bitzer, S., Karl, P., Manganiello, F., Pavoni, A., Pelosi, G., Santini, P., Schupp, J., Slaughter, F., Wachter-Zeh, A., Weger, V.: CROSS: Codes and Restricted Objects Signature Scheme - Specification Document (January 2025), https://www.cross-crypto.com/CROSS_Specification_v2.0.pdf
7. Bauer, S., Santis, F.D.: Forging dilithium and falcon signatures by single fault injection. In: Workshop on Fault Detection and Tolerance in Cryptography, FDTC 2023, Prague, Czech Republic, September 10, 2023. pp. 81–88. IEEE (2023). https://doi.org/10.1109/FDTC60478.2023.00017, https://doi.org/10.1109/FDTC60478.2023.00017
8. Baum, C., Beullens, W., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST v2: Algorithm Specifications - Version 2.0 (May 2023), https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/faest-spec-round2-web.pdf
9. Beckwith, L., Wallace, R., Mohajerani, K., Gaj, K.: A High-Performance Hardware Implementation of the LESS Digital Signature Scheme. In: Johansson, T., Smith-Tone, D. (eds.) Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023, College Park, MD, USA, August 16-18, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14154, pp. 57–90. Springer (2023). https://doi.org/10.1007/978-3-031-40003-2_3, https://doi.org/10.1007/978-3-031-40003-2_3
10. Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The SPHINCS+ Signature Framework. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. p. 2129–2146. CCS '19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3319535.3363229, https://doi.org/10.1145/3319535.3363229

11. Biasse, J.F., Micheli, G., Persichetti, E., Santini, P.: LESS is More: Code-Based Signatures Without Syndromes. In: Nitaj, A., Youssef, A. (eds.) Progress in Cryptology - AFRICACRYPT 2020. pp. 45–65. Springer International Publishing, Cham (2020)
12. Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Zaverucha, G.: Post-quantum zero-knowledge and signatures from symmetric-key primitives. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (eds.) Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017. pp. 1825–1842. ACM (2017). https://doi.org/10.1145/3133956.3133997, https://doi.org/10.1145/3133956.3133997
13. Chou, T., Persichetti, E., Santini, P.: On linear equivalence, canonical forms, and digital signatures. Des. Codes Cryptography **93**(7), 2415–2457 (Mar 2025). https://doi.org/10.1007/s10623-025-01576-1, https://doi.org/10.1007/s10623-025-01576-1
14. Czuprynko, M., Mukherjee, A., Roy, S.S.: Correlation power analysis of LESS and CROSS. Cryptology ePrint Archive, Paper 2025/839 (2025), https://eprint.iacr.org/2025/839
15. Fiat, A., Shamir, A.: How To Prove Yourself: Practical Solutions to Identification and Signature Problems. vol. 263 (03 1999). https://doi.org/10.1007/3-540-47721-7_12
16. Genêt, A.: On protecting SPHINCS+ against fault attacks. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2023**(2), 80–114 (2023). https://doi.org/10.46586/TCHES.V2023.I2.80-114, https://doi.org/10.46586/tches.v2023.i2.80-114
17. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions. J. ACM **33**(4), 792–807 (Aug 1986). https://doi.org/10.1145/6490.6503, https://doi.org/10.1145/6490.6503
18. Huang, X., Huang, Z., He, Y., Yuan, Q., Sun, C., Tibouchi, M., Yu, Y.: Faultless key recovery: Iteration-skip and loop-abort fault attacks on LESS. Cryptology ePrint Archive, Paper 2026/117 (2026), https://eprint.iacr.org/2026/117
19. Jendral, S., Dubrova, E.: Side-channel and fault injection attacks on VOLEitH signature schemes: A case study of masked FAEST. Cryptology ePrint Archive, Paper 2025/378 (2025), https://eprint.iacr.org/2025/378
20. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: PQM4: Post-quantum crypto library for the ARM Cortex-M4, https://github.com/mupq/pqm4
21. Krahmer, E., Pessl, P., Land, G., Güneysu, T.: Correction fault attacks on randomized crystals-dilithium. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2024**(3), 174–199 (2024). https://doi.org/10.46586/TCHES.V2024.I3.174-199, https://doi.org/10.46586/tches.v2024.i3.174-199
22. Melchor, C.A., Feneuil, T., Gama, N., Gueron, S., Howe, J., Joseph, D., Joux, A., Persichetti, E., Randrianarisoa, T.H., Rivain, M., Yue, D.: The Syndrome Decoding in the Head (SD-in-the-Head) Signature Scheme - Algorithm Specifications and Supporting Documentation (May 2023), https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/SDitH-spec-web.pdf
23. Meyer, C.: Matrix Analysis and Applied Linear Algebra. Other Titles in Applied Mathematics, Society for Industrial and Applied Mathematics (2000), https://books.google.co.in/books?id=HoNgdpJWnWMC
24. Mondal, P., Adhikary, S., Kundu, S., Karmakar, A.: Zkfault: Fault attack analysis on zero-knowledge based post-quantum digital signature schemes. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VIII. Lecture Notes in Computer Science, vol. 15491, pp. 132–167. Springer (2024). https://doi.org/10.1007/978-981-96-0944-4_5, https://doi.org/10.1007/978-981-96-0944-4_5

25. NewAE: Reference: V3:Tutorial A2 Introduction to Glitch Attacks (including Glitch Explorer) - ChipWhisperer Wiki, https://wiki.newae.com/V3:Tutorial_A2_Introduction_to_Glitch_Attacks_(including_Glitch_Explorer)
26. NIST: NIST Announces Additional Digital Signature Candidates for the PQC Standardization Process. Online. Accessed 26th January, 2024 (2023), https://csrc.nist.gov/news/2023/additional-pqc-digital-signature-candidates
27. Rennie, B., Dobson, A.: On stirling numbers of the second kind. Journal of Combinatorial Theory **7**(2), 116–121 (1969). https://doi.org/https://doi.org/10.1016/S0021-9800(69)80045-1, https://www.sciencedirect.com/science/article/pii/S0021980069800451
28. Schemes, N.P.Q.C.D.S.: CROSS: Codes and Restricted Objects Signature Scheme - Specification Document (Jan 2022), https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/CROSS-spec-web.pdf

A Some definition and properties on special matrices

Permutation and diagonal matrix. A square matrix $\boldsymbol{P}$ of order n is called permutation matrix if $\boldsymbol{P}[*, j] = \boldsymbol{e}_{\pi^{-1}(j)}\ \forall j \in \mathbb{Z}_n$, where π^{-1} is inverse permutation of a permutation π. A square matrix $\boldsymbol{D}$ is called diagonal matrix if all the entries of $\boldsymbol{D}$ are zero , the main diagonal, i.e., $\boldsymbol{D}$ can be written as $\boldsymbol{D} = \big(\boldsymbol{d}[0]\cdot\boldsymbol{e}_{\mathtt{Id}(0)}||\cdots||\boldsymbol{d}[n-1]\cdot\boldsymbol{e}_{\mathtt{Id}(n-1)}\big)$, where $\boldsymbol{d}$ is a vector of size n.

Monomial matrix. A square matrix $\boldsymbol{A}$ is called a monomial matrix and can be written as $\boldsymbol{A}[*, j] = \boldsymbol{u}[j]\cdot\boldsymbol{e}_{\pi^{-1}(j)}\ \forall j \in \mathbb{Z}_n$ (equivalently, $\boldsymbol{A}[\pi^{-1}(j), j] = \boldsymbol{u}[j]\ \forall j \in \mathbb{Z}_n$), where $\boldsymbol{u}$ is a vector and π^{-1} is inverse of a permutation π. In this case, we can write the monomial matrix $\boldsymbol{A}$ as the pair $(\boldsymbol{u}, \pi)$. We can think a diagonal matrix $\boldsymbol{D}$ as well as a permutation matrix $\boldsymbol{P}$ as monomial matrices which can be written as $\boldsymbol{D} = (\boldsymbol{d}, \mathtt{Id})$ and $\boldsymbol{P} = (\mathbf{1}_n, \pi)$ respectively, where $\boldsymbol{D}[*, j] = \boldsymbol{d}[j]\cdot\boldsymbol{e}_{\mathtt{Id}^{-1}(j)}$ and $\boldsymbol{P}[*, j] = 1\cdot\boldsymbol{e}_{\pi^{-1}(j)}\ \forall j \in \mathbb{Z}_n$.

Matrix multiplication with monomial matrix. Let $\boldsymbol{G} = (\boldsymbol{g}_0||\boldsymbol{g}_1||\cdots||\boldsymbol{g}_{n-1})$ be a matrix of order $k \times n$, where $\boldsymbol{g}_j = \boldsymbol{G}[*, j]\ \forall j \in \mathbb{Z}_n$. Then the multiplication of the matrices $\boldsymbol{G}$ and a monomial matrix $\boldsymbol{A} = (\boldsymbol{u}, \pi)$ is denoted by $\boldsymbol{GA}$ (or sometimes defined by $\boldsymbol{G}\cdot\boldsymbol{A}$) and is given by

$$(\boldsymbol{G}\cdot\boldsymbol{A})[*, j] = (\boldsymbol{GA})[*, j] = \boldsymbol{u}[j]\cdot\boldsymbol{g}_{\pi^{-1}(j)}\ \forall j \in \mathbb{Z}_n$$

where $\boldsymbol{u}[j]\cdot\boldsymbol{g}_{\pi^{-1}(j)}$ is the scalar multiplication of the column vector $\boldsymbol{g}_{\pi^{-1}(j)}$ by the scalar $\boldsymbol{u}[j]$. Any monomial matrix $\boldsymbol{A} = (\boldsymbol{u}, \pi)$ can be written as a multiplication of a diagonal matrix $\boldsymbol{D} = (\boldsymbol{v}, \mathtt{Id})$ and a permutation matrix $\boldsymbol{P} = (\mathbf{1}, \pi)$, where $\boldsymbol{v}[j] = \boldsymbol{u}[\pi(j)]\ \forall j \in \mathbb{Z}_n$.

Transpose and inverse of monomial matrix. Let $\boldsymbol{A} = (\boldsymbol{u}, \pi)$ be a monomial matrix of order n. Then, the transpose and inverse of $\boldsymbol{A}$ are $\boldsymbol{A}^T = \big(\boldsymbol{v}_1, \pi^{-1}\big)$ and $\boldsymbol{A}^{-1} = \big(\boldsymbol{v}_2, \pi^{-1}\big)$ respectively, where $\boldsymbol{v}_1[j] = \boldsymbol{u}[\pi(j)]$ and $\boldsymbol{v}_2[j] = \boldsymbol{u}[\pi(j)]^{-1}\ \forall j \in \mathbb{Z}_n$.

Partial monomial matrix. A matrix $\boldsymbol{B}$ of order $n \times k$ called a partial monomial matrix if if can be expressed as $\boldsymbol{B}[*, i] := \boldsymbol{v}[0]\cdot\boldsymbol{e}_{\pi^*(i)}\ \forall i \in \mathbb{Z}_k$, where $\pi^* : \mathbb{Z}_k \to \mathbb{Z}_n$ is an injective map (sometime called partial parmutation) and $\boldsymbol{v}$ is a vector of order k. Suppose $\boldsymbol{A} = (\boldsymbol{u}, \pi)$ be a monomial matrix of order n and $J = \{j_0, \cdots, j_{k-1}\}$ be a set with cardinality k, then $\boldsymbol{B} = \boldsymbol{A}[*, J]$ be partial monomial matrix of order $n \times k$ where $\boldsymbol{B}[*, i] = \boldsymbol{u}[j_i]\cdot\boldsymbol{e}_{\pi^*(i)}$ and $\pi^*(i) = \pi^{-1}(j_i)\ \forall i \in \mathbb{Z}_k$. In this case, we denote this partial monomial matrix as $\big(\boldsymbol{u}_{|J}, \pi^* = \pi_{|J}\big)$. We denote the set of all monomial, permutations, and diagonal matrices of order n over the field $\mathbb{F}_q$ as $Mono^n(q)$, $Per^n(q)$ and $Diag^n(q)$, respectively.

Some known elementary matrix operations: Let $\boldsymbol{A}$ be a matrix of order $k \times n$. Suppose we compute a matrix, say $\boldsymbol{B}$ of order $k \times n$ by applying all elementary row (/column) operations on the matrix $\boldsymbol{A}$ of order $k \times n$. In that case, we can compute the non-singular matrix, say $\boldsymbol{S}$ of order k (for column it will be of order n), such that $\boldsymbol{B} = \boldsymbol{SA}$ (for column it will be $\boldsymbol{B} = \boldsymbol{AS}$) holds. If the operation is row/column switching or row/column scalar multiplication, then the invertible matrix $\boldsymbol{S}$ will

be the special permutation matrix and monomial matrix, respectively. Any matrix $\boldsymbol{A}$ can be uniquely transferred to Reduced Row-Echelon form(RREF) by applying elementary row operations [23]. We denote $(\boldsymbol{B}, J) = \texttt{RREF}(\boldsymbol{A})$ to represent that $\boldsymbol{B}$ is the transform RREF of $\boldsymbol{A}$, where J is the set of pivot column indices of the matrix $\boldsymbol{B}$, i.e., there will be an invertible matrix $\boldsymbol{S}$ such that where the following condition holds: $\boldsymbol{B} = \boldsymbol{S}\boldsymbol{A}$ and $\boldsymbol{B}[*, J] = \boldsymbol{S}\boldsymbol{A}[*, J] = \boldsymbol{I}_k$.

B Sigma-Protocol of LESS

LESS Signature scheme follows a interactive 3 round sigma-protocol between prover and verifier. We have explained the sigma-protocol Fig. 9 that has been used in LESS-v2 signature [5].

Private Key: A monomial matrix $\mathbf{Q} \in Mono^n(q)$

Public Key: A pair $(\mathbf{G}_0, \mathbf{G})$, where $\mathbf{G}_0 \xleftarrow{\$} \mathbb{F}_q^{k \times n}$ and $(\mathbf{G}, J) = \texttt{RREF}\left(\mathbf{G}_0 (\mathbf{Q}^{-1})^T\right)$

Prover		*Verifier*
$\widetilde{\mathbf{Q}} = (\mathbf{v}, \widetilde{\pi}) \xleftarrow{\$} Mono^n(q)$		
$\left(\widetilde{\mathbf{G}}, \widetilde{J}\right) \leftarrow \texttt{RREF}\left(\mathbf{G}_0 \widetilde{\mathbf{Q}}^T\right)$		
$\widetilde{\mathbf{A}} \leftarrow \widetilde{\mathbf{G}}[*, \widetilde{J}^c]$, $\widetilde{\pi}_* \leftarrow$ Permutation of $\widetilde{\mathbf{Q}}^T[*, \widetilde{J}]$		
$\widetilde{\mathbf{B}} \leftarrow \texttt{BLIND}\left(\widetilde{\mathbf{A}}\right)$		
$\texttt{cmt} \leftarrow \texttt{HASH}\left(\texttt{CF}(\widetilde{\mathbf{B}})\right)$	$\xrightarrow{\texttt{cmt}}$	
	$\xleftarrow{\texttt{ch}}$	$\texttt{ch} \xleftarrow{\$} \{0, 1\}$
If $\texttt{ch} = 0$: $\texttt{rsp} = \widetilde{\mathbf{Q}}$		
Else:		
$\widetilde{\pi}'_* = \texttt{MonomialCompose}(\mathbf{Q}, \widetilde{\pi}_*)$		
$\texttt{rsp} = \texttt{CosetRep}(\widetilde{\pi}'_*)$	$\xrightarrow{\texttt{rsp}}$	If $\texttt{ch} = 0$:
		$\left(\widehat{\mathbf{G}}, \widehat{J}\right) \leftarrow \texttt{RREF}\left(\mathbf{G}_0 \cdot \texttt{rsp}^T\right)$
		$\widehat{\mathbf{A}} \leftarrow \widehat{\mathbf{G}}[*, \widehat{J}^c]$
		Else:
		$\left(\widehat{\mathbf{G}}, \widehat{J}'\right) \leftarrow \texttt{RREF}(\mathbf{G}[*, \texttt{rsp}] \mid \mathbf{G}[*, \texttt{rsp}^c])$
		$\widehat{\mathbf{A}} \leftarrow \widehat{\mathbf{G}}[*, \widehat{J}'^c]$
		Accept iff $\texttt{cmt} = \texttt{HASH}\left(\texttt{CF}\left(\widehat{\mathbf{A}}\right)\right)$

Fig. 8: The LESS-v2 Sigma protocol

In the LESS-v2 Sigma protocol 9, the signer needs to send either the information of the monomial matrix $\widetilde{\boldsymbol{Q}}$ or the full partial monomial matrix $\boldsymbol{Q}^T\overline{\boldsymbol{Q}}$ as response to the verifier. So, in this case, the size of the response will be either the size of the seed, say λ corresponding to the monomial matrix $\widetilde{\boldsymbol{Q}}$ or $k(\log n + \log q)$ bits, that requires

to represents the partial monomial matrix $\boldsymbol{Q}^T\overline{\boldsymbol{Q}}$. Since this protocol is interactive, LESS uses the Fiat-Shamir transformation [1] to transform into a non-interactive signature scheme. In this sigma-protocol, the soundness error of this protocol is $\frac{1}{2}$ as an dishonest prover (who does not have the secret $\boldsymbol{Q}$) always succeeds for $ch=0$. To decrease this soundness error, the prover and the verifier repeated this protocol multiple times. If the protocol runs t times, the challenge will contain t elements. Each response will be generated depending on the corresponding challenge value. So, in the standard case, the size of total responses will be $((t-w)\lambda)+(wk(\log n+\log q))$, where w is the number of non-zero values in the challenge. To decrease this response size, more specifically to reduce the part $(t-w)\times\lambda$ (this is required for $t-w$ seeds, each of length λ bits), the LESS-v1 uses an optimized technique to reduce the response size. The recent ZKFault paper [24] has shown that injecting a fault at this optimized technique's location in LESS-v1 can reveal the secret key information.

However, to reduce the signature size, LESS-v2 modifies the notions of the underlying sigma protocol by introducing some special functions (a canonical function) instead of using `LesSort` and `LexMinCol`. It also modifies the optimized technique used in LESS-v2 that helps reduce the response sizes. The modified sigma protocol 9 is written as follows:

Private Key: A monomial matrix $\mathbf{Q}\in Mono^n(q)$

Public Key: A pair $(\mathbf{G}_0, \mathbf{G})$, where $\mathbf{G}_0 \xleftarrow{\$} \mathbb{F}_q^{k\times n}$ and $(\boldsymbol{G}, J)=\texttt{RREF}\Big(\mathbf{G}_0(\mathbf{Q}^{-1})^T\Big)$

Prover		*Verifier*
$\widetilde{\mathbf{Q}}\xleftarrow{\$} Mono^n(q)$		
$\Big(\widetilde{\mathbf{G}}, \widetilde{J}\Big)\leftarrow\texttt{RREF}\Big(\mathbf{G}_0\widetilde{\mathbf{Q}}^T\Big)$		
$\widetilde{\mathbf{A}}\leftarrow\widetilde{\mathbf{G}}[*, \widetilde{J}^c]$, $\widetilde{\pi}_*\leftarrow$ Permutation of $\widetilde{\mathbf{Q}}^T[*, \widetilde{J}]$		
$\widetilde{\mathbf{B}}\leftarrow\texttt{BLIND}\Big(\widetilde{\mathbf{A}}\Big)$		
$\texttt{cmt}\leftarrow\texttt{HASH}\Big(\texttt{CF}(\widetilde{\mathbf{B}})\Big)$	$\xrightarrow{\texttt{cmt}}$	
	$\xleftarrow{\texttt{ch}}$	$\texttt{ch}\xleftarrow{\$}\{0, 1\}$
If $\texttt{ch}=0$: $\texttt{rsp}=\widetilde{\mathbf{Q}}$		
Else:		
$\widetilde{\pi}'_*=\texttt{MonomialCompose}(\mathbf{Q}, \widetilde{\pi}_*)$		
$\texttt{rsp}=\texttt{CosetRep}(\widetilde{\pi}'_*)$	$\xrightarrow{\texttt{rsp}}$	If $\texttt{ch}=0$:
		$\Big(\widehat{\mathbf{G}}, \widehat{J}\Big)\leftarrow\texttt{RREF}\big(\mathbf{G}_0\cdot\texttt{rsp}^T\big)$
		$\widehat{\mathbf{A}}\leftarrow\widehat{\mathbf{G}}[*, \widehat{J}^c]$
		Else:
		$\Big(\widehat{\mathbf{G}}, \widehat{J}'\Big)\leftarrow\texttt{RREF}(\mathbf{G}[*, \texttt{rsp}] \mid \mathbf{G}[*, \texttt{rsp}^c])$
		$\widehat{\mathbf{A}}\leftarrow\widehat{\mathbf{G}}[*, \widehat{J}'^c]$
		Accept iff $\texttt{cmt}=\texttt{HASH}\Big(\texttt{CF}\Big(\widehat{\mathbf{A}}\Big)\Big)$

Fig. 9: The LESS-v2 Sigma protocol

In the updated LESS-v2 Sigma protocol version 9, the signer sends $\texttt{CosetRep}(\widetilde{\pi}'_*)$ instead of sending the full partial monomial matrix $\boldsymbol{Q}^T\overline{\boldsymbol{Q}}$, where $\widetilde{\pi}'_*$ is a partial permutation. The $\texttt{CosetRep}(\widetilde{\pi}'_*)$ returns a n-bit string, where the non-zero bit location represents the range set of the partial permutation $\widetilde{\pi}'_*$. So, it only requires n bits instead of $k(\log n+\log q)\big(=n\big(\log\sqrt{n}+\log\sqrt{q}\big)\big)$ bits, where $k=\frac{n}{2}$. So, using the LESS-v2 sigma protocol 9, LESS reduce the signature size. Here, the new function called $\texttt{CF}$ has been introduced whose property $\texttt{CF}\Big(\widetilde{\mathbf{A}}\Big)=\texttt{CF}\Big(\mathbf{Q}_r\widetilde{\mathbf{A}}\mathbf{Q}_c\Big)$, for all $\mathbf{Q}_r\in Mono^k(q)$ and $\mathbf{Q}_c\in Mono^{n-k}(q)$ has been used for the correctness. In the honest prover side, prover generates the commitment using $\texttt{BLIND}\Big(\widetilde{\boldsymbol{A}}\Big)=\widetilde{\boldsymbol{Q}}_r\widetilde{\boldsymbol{A}}\widetilde{\boldsymbol{Q}}_c$, for some $\widetilde{\boldsymbol{Q}}_r\in Mono^k(q)$ and $\widetilde{\boldsymbol{Q}}_c\in Mono^{n-k}(q)$ and in the verifier side verifer is able to generate the matrix $\widehat{\mathbf{A}}=\mathbf{Q}_r\widetilde{\boldsymbol{A}}\boldsymbol{Q}_c$. So, the $\texttt{HASH}$ function of $\texttt{CF}\Big(\texttt{BLIND}\Big(\widetilde{\mathbf{A}}\Big)\Big)$ and $\texttt{CF}\Big(\widehat{\mathbf{A}}\Big)$ will be the same. Similar to the LESS-v1, LESS-v2 repeated this protocol t times to decrease the soundness error and uses the Fiat-Shamir transformation [1] to make this interactive protocol to the signature scheme. In this case, the size of responses should be $(t-w)\lambda+wn$ bits without the optimization technique, where w is the number of non-zero elements in the challenge. To decrease this response size, more specifically to reduce the part $(t-w)\lambda$ (this is required for $t-w$ seeds corresponding to the monomial matrices $\widetilde{\boldsymbol{Q}}$, each of length λ bits) it uses a modified version of optimized technique that used in LESS-v1. Since LESS modifies this optimization technique, that was vulnerable with respect to the ZKFault [24], this somehow prevent those Fault locations but it creates another fault locations. In this work, we will mainly analize the fault attack methods that can be applied in the new version, i.e., on LESS-v2. From now we call the LESS-v2 as LESS and the previous version as LESS-v1.

B.1 Key Generation of LESS-v2:

The $\texttt{LESS Key Generation}$ algorithm (Alg. 8) outputs the secret key $\texttt{sk}$ and the public key $\texttt{pk}$. The secret key $\texttt{sk}$ contains the seed $\texttt{seed}_{\texttt{sk}}$ that is used to generate $s-1$ seeds $\texttt{seed}_1$, $\cdots$, $\texttt{seed}_{\texttt{s}-1}$ of secret monomial matrices $\mathbf{Q}_i$, $i\in[1,\ s-1]$. The public key $\texttt{pk}$ contains the seed $\texttt{seed}_{\texttt{pk}}$ that is used to compute the public matrix $\mathbf{G}_0$. Also, the public key $\texttt{pk}$ contains $s-1$ public matrices $\mathbf{G}_i$, $i\in[1,\ s-1]$, where each matrix $\mathbf{G}_i$ is generated using the secret monomial matrix $\mathbf{Q}_i$ and the public matrix $\mathbf{G}_0$.

B.2 Less Verification

$\texttt{LESS Verify}$ algorithm (Alg. 9) takes the $\texttt{msg}$, the corresponding signature $\tau'=\Big\{\texttt{cmt}',\ \texttt{salt}',\ \texttt{path}',\ \{\texttt{rsp}'_i\}_{i\in\mathbb{Z}_w}\Big\}$ and the public key $\texttt{pk}=(\texttt{seed}_{\texttt{pk}},\ \mathbf{G}_1,\ \cdots,\ \mathbf{G}_{s-1})$ as inputs and returns True if the given message and signature pair is valid, otherwise it will return false.

This algorithm first generates the fixed weight digest $\mathbf{b}'$ corresponding to the given commitment $\texttt{cmt}'$. Then, depending on each values of the digest vector $\mathbf{b}'$, it will recompute the commitment $\texttt{cmt}''$ from the remaining information $\texttt{salt}'$, $\texttt{path}'$ and $\texttt{rsp}'=\{\texttt{rsp}'_i\}_{i\in\mathbb{Z}_w}$ and message $\texttt{msg}$. If all of this information is valid, then the

Algorithm 8 LESS-v2 Key Generation [11,9]

Input: Private seed $\mathtt{seed}_{\mathtt{sk}}$
Output: secret key $\mathtt{sk}$ and public key $\mathtt{pk}$

$\mathtt{state}_{\mathtt{sk}} \leftarrow \mathtt{XOF.init}(\mathtt{seed}_{\mathtt{sk}})$
$\mathtt{seed}_{\mathtt{pk}} \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$\mathbf{G}_0 \leftarrow \mathtt{SampleGenerator}(\mathtt{seed}_{\mathtt{pk}})$
$(\mathtt{seed}_1, \mathtt{seed}_2, \cdots, \mathtt{seed}_{s-1}) \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
for $i = 1, 2, \cdots, s-1$ **do**
    $\mathbf{Q}_i \leftarrow \mathtt{SampleMonomial}(\mathtt{seed}_i)$
    $\mathbf{Q}_i^{-1} \leftarrow \mathtt{Inverse\_Monomial}(\mathbf{Q}_i)$
    $(\mathbf{G}_i, \mathtt{pivot\_column}) \leftarrow \mathtt{RREF}(\mathbf{G}_0(\mathbf{Q}_i^{-1})^T)$
$\mathtt{sk} \leftarrow \mathtt{seed}_{\mathtt{sk}}$
$\mathtt{pk} \leftarrow (\mathtt{seed}_{\mathtt{pk}}, \mathbf{G}_1, \cdots, \mathbf{G}_{s-1})$
Return $(\mathtt{sk}, \mathtt{pk})$

recomputed commitment $\mathtt{cmt}''$ will be equal to the given commitment $\mathtt{cmt}'$. In this case, it will returns true. Otherwise, it will return false, which represents that all of this information is not valid.

Algorithm 9 LESS Verify

Input: Message $\mathtt{msg}$, public key $\mathtt{pk} = (\mathtt{seed}_{\mathtt{pk}}, \mathbf{G}_1, \cdots, \mathbf{G}_{s-1})$ and signature $\tau' = \left\{\mathtt{cmt}', \mathtt{salt}', \mathtt{path}', \{\mathtt{rsp}_i'\}_{i \in \mathbb{Z}_w}\right\}$.
Output: True (Accept) or False (reject).

$\mathbf{G}_0 \leftarrow \mathtt{SampleGenerator}(\mathtt{seed}_{\mathtt{pk}})$
$\mathbf{b}' = (\mathbf{b}'[0], \cdots, \mathbf{b}'[t-1]) \leftarrow \mathtt{SampleChallenge}(\mathtt{cmt}')$
for $i = 0$ to $t-1$ **do**
    if $\mathbf{b}'[i] \neq 0$ **then**
        $\mathbf{U}'[i] = 1$
    else
        $\mathbf{U}'[i] = 0$
For $\mathbf{b}'[i] = 0$, $\mathtt{eseed}_i' \leftarrow \mathtt{RebuildGGM}(\mathtt{path}', \mathbf{U}', \mathtt{salt}')$
for $i = 0, \cdots, t-1$ **do**
    if $\mathbf{b}'[i] = 0$ **then**
        $\widehat{\mathbf{Q}}_i \leftarrow \mathtt{SampleMonomial}(\mathtt{eseed}_i' || \mathtt{salt}' || i)$
        $\left(\widehat{\mathbf{G}}_i, \mathtt{pivot\_column}'\right) \leftarrow \mathtt{RREF}(\mathbf{G}_0 \widehat{\mathbf{Q}}_i^T)$
        $\widehat{\mathbf{A}}_i = \widehat{\mathbf{G}}_i[*, \mathtt{pivot\_column}']$
    else
        if $\mathtt{CheckCosetRep}(\mathtt{rsp}_i') =$ false **then return** false
        Set $\mathbf{G}_1' :=$ columns of $\mathbf{G}_{\mathbf{b}[i]}$ indexed by $\mathtt{rsp}_i'$
        Set $\mathbf{G}_2' :=$ columns of $\mathbf{G}_{\mathbf{b}[i]}$ not indexed by $\mathtt{rsp}_i'$
        $\widehat{\mathbf{A}}_i \leftarrow \mathbf{G}_1'^{-1} \cdot \mathbf{G}_2'$
    $\widehat{\boldsymbol{C}}_i \leftarrow \mathtt{CF}\left(\widehat{\boldsymbol{A}}_i\right)$
$\mathtt{cmt}'' \leftarrow \mathtt{HASH}\left(\widehat{\boldsymbol{C}}_0 || \cdots || \widehat{\boldsymbol{C}}_{t-1} || \mathtt{salt}' || \mathtt{msg}\right)$
if $\mathtt{cmt}' = \mathtt{cmt}''$ **then return** true
else
    return false

B.3 Parameters

The parameter sets of LESS-v2 [4] corresponding to three security levels are given in Table 2.

Table 2: Parameter set of LESS-v2 [5] for different security levels

Security level	Parameter set	Parameters						Public key	Signature (τ)	
		n	k	q	t	w	s	pk(B)	Worst Case	Avg Case
1	LESS-252-192	252	126	127	192	36	2	13940	2625	2289
	LESS-252-68				68	42	4	41788	1825	1313
	LESS-252-45				45	34	8	97484	1329	1745
3	LESS-400-220	400	200	127	220	68	2	35074	6329	5585
	LESS-400-102				102	61	4	105174	4131	3867
5	LESS-548-345	548	274	127	345	75	2	65793	10680	9464
	LESS-548-137				137	79	4	197315	7436	7116

C Binary decision tree generation in LESS-v1

Now, we will explain the seeds corresponding to the ephemeral monomial matrices generation. This ephemeral seed generation is different from the previous version of LESS (i.e., LESS-v1).

In the earlier version, it generates a complete seed tree with all leaf nodes and then takes the t leftmost leaf nodes to generate the ephemeral monomial matrices. So, some nodes have not been used if $t \neq 2^r$, where t is the challenge length. For example: for $t=45$, the GGM tree of LESS-v1 should be the complete tree with 64 leaf nodes in Fig.10. It takes the left most 45 leaf nodes $\mathtt{seed}'_{63},\cdots,\mathtt{seed}'_{107}$ (marked as blue colored in Fig. 10) as seeds of the ephemeral matrices. But the remaining leaf nodes i.e., $\mathtt{seed}'_{108},\cdots,\mathtt{seed}'_{126}$ (marked as yellow colored in Fig. 10) have not used. These remaining nodes and their parents occupy extra memory. However, to reduce

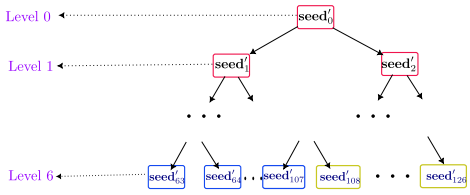


Fig. 10: GGM tree structure of LESS-v1 with $t=45$

the extra memory, this version, LESS-v2, generates an incomplete seed tree for $t \neq 2^r$ so that there would not be any computation cost for the used node generation.

D Some supporting algorithms of LESS-v2

Algorithm 10 `BuildGGM`

Input: A seed $\texttt{seed}_{\texttt{tree}} \in \{0,1\}^{\lambda}$ and a salt, $\texttt{salt} \in \{0,1\}^{2\lambda}$, where λ is security parameter.
Output: t seeds $(\texttt{eseed}_0, \cdots, \texttt{eseed}_{t-1}) \in \{0,1\}^{t\lambda}$ of ephimeral matrices.

$\texttt{seed}'_0 \leftarrow \texttt{seed}_{\texttt{tree}}$
$\texttt{index} \leftarrow 0$
for $\texttt{level} = 0$ to $T-1$ **do**
  for $i = 0$ to $\texttt{n}\ell[\texttt{level}] - \ell\ell[\texttt{level}]$ **do**
    $j \leftarrow \texttt{index} + \texttt{i}$
    $\texttt{left child} \leftarrow 2j + 1 - \texttt{off}[\texttt{level}]$
    $\texttt{right child} \leftarrow \texttt{left child} + 1$
    $\texttt{state} := \texttt{XOF-INIT}(\texttt{seed}'_j \mid \texttt{salt} \mid j)$
    $(\texttt{seed}'_{\texttt{left child}}, \texttt{seed}'_{right\ child}) \leftarrow \texttt{XOF}(\texttt{state}, 2\lambda)$
  $\texttt{index} \leftarrow \texttt{index} + \texttt{n}\ell[\texttt{level}]$
Return $\texttt{SeedLeaves}(\texttt{seed}')$

Algorithm 11 `SeedLeaves`

Input: A GGM tree $\texttt{seed}'$.
Output: t seeds $(\texttt{eseed}_0, \cdots, \texttt{eseed}_{t-1}) \in \{0,1\}^{t\lambda}$ of ephimeral matrices.

$\texttt{k} \leftarrow 0$
for $\texttt{i} = 0$ to $\texttt{T}'$ **do**
  for $\texttt{j} = 0$ to $\texttt{cons\_leaves}[\texttt{i}]$ **do**
    $\texttt{eseed}_k \leftarrow \overline{\texttt{seed}'}_{\texttt{lsi}[\texttt{i}]+\texttt{j}}$
    $\texttt{k} = \texttt{k} + 1$
Return $(\texttt{eseed}_0, \cdots, \texttt{eseed}_{t-1})$

E Signature Forgery

Here, we have provided the signature forgery in `LESS Signature Forge` algorithm (Alg. 12). This algorithm takes a message `msg`, all secret permutations $\pi_1, \cdots, \pi_{s-1}$ and the public key $\texttt{pk} = \left(\texttt{seed}_{\texttt{pk}}, \mathbf{G}_1, \cdots, \mathbf{G}_{s-1}\right)$ as input an return the signature τ of the message `msg` as output. To compare this algorithm with the actual LESS signature (LESS-v2), we first write the LESS signature and then strike through the operations that are dependent on the secret $\texttt{seed}_{\texttt{sk}}$ and replace the new operations that depend on the public key $\texttt{seed}_{\texttt{pk}}$ and the secret permutations. We denote the new operations in blue.

Algorithm 12 LESS Signature Forge

Input: Message msg and permutation of secret monomial matrices $\pi_1, \cdots, \pi_{s-1}$ and the public key $\mathtt{pk} = (\mathtt{seed}_{\mathtt{pk}}, \mathbf{G}_1, \cdots, \mathbf{G}_{s-1})$.
Output: The valid signature τ of the message msg.

$\mathtt{state}_{\mathtt{sk}} \leftarrow \mathtt{XOF.init}(\mathtt{seed}_{\mathtt{sk}})$
$\mathtt{seed}_{\mathtt{pk}} \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$\mathbf{G}_0 \leftarrow \mathtt{SampleGenerator}(\mathtt{seed}_{\mathtt{pk}})$
$(\mathtt{seed}_1, \mathtt{seed}_2, \cdots, \mathtt{seed}_{s-1}) \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$\mathtt{salt} \xleftarrow{\$} \{0,1\}^{2\lambda}$
$\mathtt{seed}_{\mathtt{tree}} \xleftarrow{\$} \{0,1\}^{\lambda}$ // Previously it was $\mathtt{seed}_{\mathtt{tree}} \leftarrow \mathtt{XOF}(\mathtt{state}_{\mathtt{sk}})$
$(\mathtt{seed}'_0, \cdots, \mathtt{seed}'_{2t-1}) \leftarrow \mathtt{BuildGGM}(\mathtt{seed}_{\mathtt{tree}}, \mathtt{salt})$
$(\mathtt{eseed}_0, \ldots, \mathtt{eseed}_{t-1}) \leftarrow \mathtt{SeedLeaves}(\mathtt{seed}')$
$\mathtt{state}_{\mathtt{blind}} \leftarrow \mathtt{XOF.init}(\mathtt{XOF}(\mathtt{state}_{\mathtt{sk}}))$
for $i = 0, \cdots, t-1$ **do**
    $\widetilde{\mathbf{Q}}_i = (\mathbf{v}_i, \widetilde{\pi}_i) \leftarrow \mathtt{SampleMonomial}(\mathtt{eseed}_i || \mathtt{salt} || i)$
    $\left(\widetilde{\boldsymbol{G}}_i, \widetilde{J}\right) \leftarrow \mathtt{RREF}(\mathbf{G}_0 \widetilde{\boldsymbol{Q}}_i^T)$
    $\widetilde{\pi}_i^* = \left(\widetilde{\pi}^{-1}\right)_{|J}$
    $\mathbf{A}_i = \widetilde{\mathbf{G}}_i[*, J^c]$
    $\mathbf{B}_i \leftarrow \mathtt{BLIND}(\mathtt{state}_{\mathtt{blind}}, \mathbf{A}_i)$
    $\mathbf{C}_i \leftarrow \mathtt{CF}(\mathbf{A}_i)$ //Previously it was $\mathbf{C}_i \leftarrow \mathtt{CF}(\mathbf{B}_i)$
$\mathtt{cmt} \leftarrow \mathtt{HASH}(\mathbf{C}_0, \ldots, \mathbf{C}_{t-1} || \mathtt{salt} || \mathtt{msg})$
$(\mathbf{b}[0], \cdots, \mathbf{b}[t-1]) \leftarrow \mathtt{SampleChallenge}(\mathtt{cmt})$
for $i = 0$ to $t-1$ **do**
    if $\mathbf{b}[i] \neq 0$ **then**
        $\mathbf{U}[i] = 1$
    else
        $\mathbf{U}[i] = 0$
$\mathtt{path} \leftarrow \mathtt{GGMPath}(\mathtt{seed}', \mathbf{U})$
$k = 0$
for $i = 0, \cdots, t-1$ **do**
    if $\mathbf{b}[i] \neq 0$ **then**
        $\mathbf{Q}_{\mathbf{b}[i]} \leftarrow \mathtt{SampleMonomial}(\mathtt{seed}_{\mathbf{b}[i]})$
        $\pi^{*'} \leftarrow \mathtt{MonoCom\_new}\left(\pi_{\mathbf{b}[i]}, \widetilde{\pi}_i^*\right)$ $\mathtt{MonomialCompose}\left(\mathbf{Q}_{\mathbf{b}[i]}, \widetilde{\pi}_i^*\right)$
        $\mathtt{rsp}_k \leftarrow \mathtt{CosetRep}\left(\pi^{*'}\right)$, $k = k+1$
$\tau \leftarrow \left\{\mathtt{cmt}, \mathtt{salt}, \mathtt{path}, \{\mathtt{rsp}_i\}_{i \in \mathbb{Z}_w}\right\}$
Return τ